

Open-Source Software (OSS) Supply Chain Attack Campaign Analysis

2024–2026 | Multi-Ecosystem, Multi-Actor Intelligence Report

Classification	TLP:CLEAR — Public, Unlimited Distribution
Report Version	1.0 (Initial Release)
Report Date	22 May 2026
Status	ACTIVE — Multiple campaigns ongoing
Prepared By	Douglas Mun with AI analyst agents
Primary CVEs	CVE-2026-33634 (Trivy), CVE-2026-45321 (TanStack)
Threat Actors	TeamPCP (UNC6780), Lazarus Group, PCPJack, and others

TABLE OF CONTENTS

- [1. DISCLAIMER](#)
- [2. EXECUTIVE SUMMARY](#)
- [3. BOTTOM LINE UP FRONT \(BLUF\)](#)
- [4. THE 5 Ws AND H](#)
- [5. DETAILED TIMELINE \(2024–2026\)](#)
- [6. DETAILED SEQUENCE OF EVENTS BY PHASE AND ACTOR](#)
 - [6.1 TeamPCP Campaign — Phases 1–9](#)
 - [6.2 Bitwarden CLI npm Compromise \(April 2026\)](#)
 - [6.3 XPACK Attack \(4 February 2026\)](#)
 - [6.4 axios npm Compromise \(31 March 2026\)](#)
 - [6.5 Lazarus Group — Contagious Interview / graphalgo Campaign \(May 2025–May 2026\)](#)
 - [6.6 Mini Shai-Hulud — SAP + PyTorch Lightning + intercom-client \(29–30 April 2026\)](#)
 - [6.7 TanStack Attack \(11 May 2026\) — CVE-2026-45321](#)
 - [6.8 Nx Console VS Code Extension Trojanization \(18 May 2026\)](#)
 - [6.9 GitHub Internal Breach \(19–21 May 2026\)](#)
 - [6.10 PCPJack — Rival Worm Evicting TeamPCP \(Discovered 28 April 2026\)](#)
 - [6.11 TeamPCP × BreachForums \\$1,000 Supply Chain Attack Contest \(14 May 2026\)](#)
- [7. TECHNICAL DETAILS OF MAJOR CAMPAIGNS](#)
 - [7.1 TeamPCP / SANDCLOCK Technical Arsenal](#)
 - [7.2 Lazarus Technical Arsenal](#)
- [8. CHARTS, DIAGRAMS, AND INDICATORS](#)
 - [8.1 Worm Generation Evolution](#)
 - [8.2 Threat Actor Operational Model Comparison](#)
- [9. INDICATORS OF COMPROMISE \(IOCs\)](#)
 - [9.1 TeamPCP / CanisterWorm / SANDCLOCK IOCs \(Phases 1–6\)](#)
 - [9.2 Mini Shai-Hulud / TanStack IOCs \(CVE-2026-45321\) — Gen 4](#)
 - [9.3 Lazarus / Contagious Interview IOCs](#)
 - [9.4 Shai-Hulud Gen 1/2 IOCs](#)
 - [9.5 Context.ai / Vercel \(April 2026\) — Unattributed](#)
 - [9.6 PCPJack IOCs](#)
- [10. MITRE ATT&CK MAPPING](#)
 - [10.1 TeamPCP / UNC6780 \(Phases 1–9 including Mini Shai-Hulud\)](#)
 - [10.2 Lazarus \(Contagious Interview\) ATT&CK](#)
 - [10.3 PCPJack ATT&CK Coverage](#)
- [11. ATTRIBUTION ASSESSMENT](#)
 - [11.1 TeamPCP / UNC6780 / SANDCLOCK](#)
 - [11.2 Lazarus Group / DPRK \(Contagious Interview / graphalgo / Sapphire Sleet\)](#)
 - [11.3 Shai-Hulud \(Worm Lineage — Gen 1 and Gen 2\)](#)
 - [11.4 Context.ai / Vercel \(April 2026\)](#)
 - [11.5 XPACK Attack / Ultralytics / tj-actions](#)
- [12. SUPPLY CHAIN ATTACK LINEAGE AND EVOLUTION](#)

- [12.1 Technique Evolution by Campaign](#)
- [12.2 Interval Compression](#)
- [13. THREAT ACTOR PROFILES](#)
 - [13.1 TeamPCP / UNC6780](#)
 - [13.2 Lazarus Group / DPRK](#)
 - [13.3 PCPJack \(Possible ex-TeamPCP Affiliate\)](#)
- [14. THREAT ACTOR'S OWN DEFENSIVE GUIDANCE AND ANALYST ASSESSMENT](#)
 - [14.1 TeamPCP's Five Defensive Recommendations](#)
 - [14.2 Analyst Assessment](#)
- [15. IMPACT ASSESSMENT AND COMPARATIVE ANALYSIS](#)
 - [15.1 TeamPCP Campaign vs. SolarWinds vs. Log4j](#)
 - [15.2 Why the GitHub Breach Represents a Qualitative Escalation](#)
 - [15.3 Victim Risk Tiers](#)
- [16. RECOMMENDATIONS](#)
 - [16.1 Immediate Actions \(0–48 Hours\)](#)
 - [16.2 Short-Term Actions \(1–2 Weeks\)](#)
 - [16.3 Medium-Term Actions \(1–3 Months\)](#)
- [17. WEAKNESSES BYPASSED — PRACTITIONER ANALYSIS](#)
 - [WB-1 — Mutable Git Tags as Version Pinning](#)
 - [WB-2 — pull_request_target as Trusted Execution Context](#)
 - [WB-3 — Official Namespace as Trust Signal](#)
 - [WB-4 — SLSA Build Level 3 Provenance as Trust Anchor](#)
 - [WB-5 — GitHub Actions Cache Isolation](#)
 - [WB-6 — IDE Extension Safety Signals](#)
 - [WB-7 — Credential Isolation in CI/CD Runner Memory](#)
 - [WB-8 — Non-Atomic Credential Rotation During Active Breach](#)
 - [WB-9 — Security Tooling as Trusted Execution Context](#)
- [18. LESSONS LEARNED](#)
 - [LL-1: Every Trust Signal Has Now Been Defeated at Least Once](#)
 - [LL-2: The Tools You Trust Most Are the Highest-Value Targets](#)
 - [LL-3: Credential Rotation Fails Unless It Is Atomic](#)
 - [LL-4: The Worm's Source Code Being Public Is Structural, Not an Incident](#)
 - [LL-5: Decentralized and Blockchain C2 Makes Takedown Ineffective](#)
 - [LL-6: IDE Extensions Are Production-Grade Attack Surface](#)
 - [LL-7: Rival Actors Competing on Your Victim Population Is a New Threat Class](#)
 - [LL-8: Gamified Attacks Scale Beyond What Any Single Actor Could Achieve](#)
- [19. CALL TO ACTION — A MESSAGE TO THE OSS ECOSYSTEM](#)
 - [To OSS Maintainers](#)
 - [To Platform Operators \(GitHub, npm, PyPI, Docker Hub, VS Code Marketplace\)](#)
 - [To Enterprise Consumers of OSS](#)
 - [To Security Researchers and the CTI Community](#)
 - [The Bottom Line](#)
- [20. SUMMARY](#)
- [21. REFERENCES](#)

1. DISCLAIMER

This report is classified TLP:CLEAR and approved for unlimited distribution. It is based entirely on open-source intelligence (OSINT) from vendor advisories, government statements (CISA, CERT-EU, GTIG), academic research, and security firm publications. No classified or non-public information has been used.

The OSS supply chain attack landscape is rapidly evolving. Attribution statements are marked with confidence levels: **CONFIRMED** (official disclosure or regulatory verification), **STRONGLY REPORTED** (credible vendor coverage without primary self-disclosure), or **CLAIMED / PARTIALLY CORROBORATED** (primarily extortion-side, tracker-based, or single-source reporting without corroboration). Where evidence supports inference rather than proof, this is explicitly noted.

Multiple threat actors operate in parallel. This report does not conflate unrelated incidents and preserves distinct attribution chains even where attacks occur contemporaneously or use similar techniques.

2. EXECUTIVE SUMMARY

Between December 2024 and May 2026, at least five distinct threat actor clusters carried out high-impact supply chain attacks across npm, PyPI, GitHub Actions, Docker Hub, OpenVSX, the Visual Studio Marketplace, and their associated ecosystems. The aggregate confirmed impact includes:

- **1,700+ malicious packages** across multiple language ecosystems
- **10,000+ affected repositories** (directly poisoned or indirectly exposed)
- **1,000+ confirmed affected organizations** (Mandiant CTO assessment, March 2026)
- **5,000–10,000 estimated affected organizations** (secondary projections)
- **Multiple confirmed government-level breaches** (European Commission via CERT-EU; Cisco strongly reported)
- **GitHub itself breached** (May 2026; ~3,800 internal repositories exfiltrated via poisoned VS Code extension; GitHub confirmed figure as 'directionally consistent')
- **Credential theft at scale:** 33,185 unique secrets exposed in Shai-Hulud 2.0 alone; 300+ GB in TeamPCP campaign
- **SLSA Build Level 3 provenance defeated** for the first time (TanStack attack, May 2026)

Key Insight: This is not a series of isolated incidents. It is a self-reinforcing ecosystem of evolving attack techniques, credential mutualization, and cross-actor learning. Each campaign teaches the next: tag-poisoning proved effective (tj-actions 2025); AI-assisted reconnaissance worked (s1ngularity 2025); self-propagating worms scaled autonomously (Shai-Hulud 2025); cross-ecosystem pivoting is feasible (TeamPCP 2026); credential-free initial access via OIDC/cache poisoning is now proven (Mini Shai-Hulud/TanStack 2026). The interval between major campaigns has compressed from years (SolarWinds 2020, Codecov 2021) to weeks.

Critical Assessments:

- **TeamPCP (UNC6780/SANDCLOCK/Mini Shai-Hulud):** Financially motivated; demonstrated ability to compromise security tooling, IDE extensions, and the GitHub platform itself; worm source code published publicly (May 12, 2026), enabling copycat activity.
- **Lazarus / DPRK (Contagious Interview / graphalgo / Sapphire Sleet):** State-sponsored; fake-recruiter infrastructure; responsible for the Axios compromise (~100M weekly downloads); 1,700+ malicious packages across 5 ecosystems.
- **Shai-Hulud worm lineage (attributed to TeamPCP by multiple vendors):** Four generations; September 2025 through May 2026; first npm self-propagating worm; Gen 4 (Mini Shai-Hulud) produced the first valid SLSA L3-attested malicious packages.
- **tj-actions / Ultralytics (unattributed):** Proof-of-concept attacks demonstrating tag poisoning and GitHub Actions script injection as viable vectors.

3. BOTTOM LINE UP FRONT (BLUF)

The open-source software supply chain is now a primary, continuously exploited attack surface — not a peripheral risk. As of May 22, 2026:

- **TeamPCP has compromised GitHub itself.** The breach of approximately 3,800 internal GitHub repositories via a poisoned VS Code extension is the highest-impact downstream event in the campaign. Organizations that rely on GitHub's platform integrity now have direct exposure.
- **SLSA provenance is no longer a trust anchor in isolation.** The TanStack attack (May 11, 2026) produced the first malicious npm packages that carried valid SLSA Build Level 3 attestations by executing them from within the compromised legitimate CI runner. Attestation must be combined with pipeline access controls, not treated as a substitute for them.
- **Credential-free initial access is proven.** Mini Shai-Hulud accessed TanStack's publishing pipeline via GitHub Actions cache poisoning + OIDC token theft — no stolen maintainer credentials required. Organizations that have hardened against credential theft but not OIDC token abuse remain exposed.
- **The worm's source code is now public.** TeamPCP published Mini Shai-Hulud's source on May 12, 2026. Copycat activity has begun. Future attacks attributed to "TeamPCP" may be carried out by independent actors using the same toolchain.
- **IDE extensions are now a primary attack vector.** The Nx Console trojanization (2.2M installs, 18 minutes) directly led to the GitHub breach. Organizations without extension allowlisting governance are exposed.
- **Immediate priorities:** Rotate all credentials accessible from any CI/CD pipeline that ran affected packages; audit IAM for unauthorized key creation; block C2 infrastructure (see Section 9); restrict OIDC token scopes in GitHub Actions; implement extension allowlisting.

4. THE 5 Ws AND H

WHO?

- **TeamPCP** (DeadCatx3, PCPcat, PersyPCP, ShellForce, CipherForce; formally UNC6780 per GTIG): Financially motivated cloud-native cybercriminal operation; 2–4 person team, likely Western hemisphere (UTC-4 to UTC-9); active December 2025–May 2026; operates Shai-Hulud worm framework.
- **Lazarus Group** (APT38, Sapphire Sleet, BlueNoroff, Famous Chollima, Shifty Corsair): DPRK state-sponsored; running the Contagious Interview/graphalgo fake-recruiter campaign; also attributed to the Axios compromise; active May 2025–May 2026.
- **UNC6426 / s1ngularity actors**: Unattributed; August 2025 Nx build system compromise; AI-assisted reconnaissance; possible credential-pool predecessor to Shai-Hulud.
- **Vect (ransomware group)**: Announced partnership with TeamPCP (Akamai, strongly reported); may enable ransomware follow-on operations using stolen credentials.

WHAT?

Supply chain poisoning across six distinct ecosystems (npm, PyPI, GitHub Actions, Docker, OpenVSX, Visual Studio Marketplace); credential harvesting from CI/CD pipelines, developer machines, and cloud environments; self-propagating worms (CanisterWorm, Mini Shai-Hulud); geopolitically targeted destructive payloads (Iran wiper); AI-assisted reconnaissance; cross-ecosystem pivoting; OIDC token theft; IDE extension trojanization; platform-level breach (GitHub).

WHEN?

- **December 2024**: Ultralytics PyPI (cryptominer)
- **March 2025**: tj-actions tag-poisoning; 23,000 affected repos
- **May 2025 onward**: Lazarus graphalgo / Contagious Interview begins
- **August 2025**: s1ngularity / Nx (AI-assisted)
- **Sep–Nov 2025**: Shai-Hulud Gen 1 and Gen 2 (first npm self-propagating worm)
- **December 2025**: Operation PCPcat (60,000+ servers)
- **Feb 4, 2026**: XPACK attack (npm paywall extortion)
- **Feb–Mar 2026**: TeamPCP Phases 1–6 (Trivy, Checkmarx, LiteLLM, Telnyx)
- **Mar 31, 2026**: axios compromise (Lazarus/Sapphire Sleet)
- **Mar–Apr 2026**: TeamPCP downstream exploitation; European Commission breach
- **Apr 2026**: Bitwarden CLI npm compromised
- **Apr 19–20, 2026**: Context.ai/Vercel OAuth breach (unattributed)
- **Apr 29–30, 2026**: Mini Shai-Hulud debut: SAP + PyTorch Lightning + intercom-client
- **May 11, 2026**: TanStack attack (CVE-2026-45321); first SLSA L3 provenance defeat; 373 malicious versions; OpenAI impacted
- **May 12, 2026**: Mini Shai-Hulud source code published publicly; copycat activity begins
- **May 18, 2026**: Nx Console VS Code extension trojanized (2.2M installs, 18 minutes)

- **May 7, 2026:** PCPJack rival worm disclosed; targets same victim population as TeamPCP; evicts TeamPCP artifacts before stealing credentials
- **May 11–14, 2026:** TanStack attack (CVE-2026-45321); TeamPCP + BreachForums launch \$1,000 Monero supply chain attack competition using open-sourced Shai-Hulud tooling
- **May 18, 2026:** Nx Console VS Code extension trojanized (2.2M installs; 18 minutes)
- **May 19–21, 2026:** GitHub internal breach (~3,800 repos; GitHub confirmed as 'directionally consistent'); LAPSUS\$ joint data sale; confirmed May 21

WHERE?

Global. Registries: npm, PyPI, RubyGems, crates.io, Maven Central, PHP Composer, Go modules, OpenVSX, and Visual Studio Marketplace. Confirmed government impact: European Commission (CERT-EU). Platform-level breach: GitHub (confirmed). High-profile commercial victims: OpenAI, Mistral AI, UiPath, OpenSearch, Mercor, ownCloud.

WHY?

- **Financial gain:** Credential theft; cloud intrusion; cryptomining; extortion
- **Access brokering:** Stolen credentials sold to ransomware operators (Vect partnership, strongly reported)
- **Geopolitical:** Iran-targeted wiper; DPRK state-directed access (Lazarus)
- **Reputation/signaling:** Public taunting; worm source code release; campaign branding
- **Strategic intelligence:** GitHub source code exfiltration enables offline vulnerability research

HOW?

- **Initial Access:** GitHub Actions tag poisoning; pull_request_target exploit; stolen maintainer credentials; fake recruiter social engineering; cache poisoning + OIDC token theft (credential-free, TanStack); VS Code extension trojanization
- **Execution:** Malicious postinstall/preinstall scripts; .pth auto-execution; GitHub Actions; OpenVSX/VS Marketplace plugins; WAV steganography; OIDC token exchange
- **Persistence:** systemd user services; .pth files; Kubernetes DaemonSets; AWS IAM secondary key creation; self-hosted GitHub Actions runners; Windows Startup folder; LaunchAgents (macOS)
- **Credential Theft:** /proc/[pid]/mem scraping; 50+ filesystem paths; cloud metadata APIs; Kubernetes tokens; 1Password vaults; Claude Code configs; encrypted AES-256-CBC + RSA-4096 exfiltration
- **Propagation:** npm worm (deploy.js + findNpmTokens()); ICP blockchain C2; Session decentralized messaging C2; OIDC-driven self-publishing
- **Monetization:** Exfiltration; TruffleHog credential validation; AWS control-plane reconnaissance; data theft; extortion; underground credential markets

5. DETAILED TIMELINE (2024–2026)

Date	Event	Actor	Ecosystems	Scale
4 Dec 2024	Ultralytics YOLO PyPI compromise — GitHub Actions pull_request_target + branch-name script injection; XMRig cryptominer. 4 malicious versions.	Unattributed	PyPI	~60M total project downloads
14–15 Mar 2025	tj-actions/changed-files tag poisoning (CVE-2025-30066); cascade from reviewdog/action-setup (CVE-2025-30154); 23,000+ repos affected.	Unattributed	GitHub Actions	23,000+ repos; 218 confirmed secret exposures
May 2025 onward	Lazarus graphalgo / Contagious Interview campaign begins. Fake recruiter profiles on LinkedIn, Facebook, Reddit; malicious packages delivered as "coding assignments."	Lazarus (APT38)	npm, PyPI, Go, Rust, PHP	1,700+ malicious packages by May 2026
22 Aug 2025	s1ngularity / Nx compromise — pull_request_target exploit; AI CLI tool weaponization; 1,079 systems, 1,000+ valid credentials, AWS admin in 72 hrs.	UNC6426	npm	1,079 systems; 2,349+ credentials
Sep 2025	Shai-Hulud Gen 1 (original worm). First self-propagating npm worm; steals maintainer tokens, auto-republishes poisoned versions. Patient zero: @ctrl/tinycolor v4.1.1.	TeamPCP (retrospectively attributed by multiple vendors)	npm	Initial wave: hundreds of packages
Nov 2025	Shai-Hulud Gen 2 ("SHA1-Hulud" / "The Second Coming"). Data wiping, Bun obfuscation, pre-install execution, botnet secret sharing, dead man's switch.	TeamPCP	npm, GitHub	700+ packages; 33,185 secrets; 3,760 valid
Dec 2025	Operation PCPcat — TeamPCP cloud infrastructure exploitation; 60,000+ servers cryptomining.	TeamPCP	Cloud-native	60,000+ servers
Early 2026	Shai-Hulud Gen 3 (SANDWORM_MODE). Adaptive CI/CD enumeration before propagation; evolved response to detection techniques.	TeamPCP	npm	Active in early 2026
4 Feb 2026	XPACK ATTACK — npm packages by "dev.chandra_bose" use HTTP 402 "Payment Required" responses to demand 0.1 USDC/ETH before completing the install; collect GitHub usernames and device fingerprints.	Unattributed	npm	Multiple packages
20 Feb 2026	hackerbot-claw GitHub account created. AI-powered autonomous attack bot; scans thousands of repos for pull_request_target misconfigurations.	TeamPCP	GitHub	Scans thousands of repos
27–28 Feb 2026	Phase 1 — Trivy initial access. hackerbot-claw exploits apidiff.yaml pull_request_target; exfiltrates ORG_REPO_TOKEN with write access to 33+ Aqua Security repos.	TeamPCP	GitHub Actions	33+ repos

Date	Event	Actor	Ecosystems	Scale
1 Mar 2026	Aqua Security discloses initial breach. Non-atomic credential rotation; attacker retains access.	TeamPCP	GitHub	33+ repos
19 Mar 2026 ~17:43 UTC	Phase 2 — Trivy supply chain injection. Force-push 76/77 trivy-action tags + all setup-trivy tags; trigger malicious Trivy v0.69.4 to GitHub, Docker Hub, GHCR, ECR, deb/rpm, get.trivy.dev. CISA KEV: CVE-2026-33634.	TeamPCP	GitHub Actions, Docker	18:22–21:42 UTC v0.69.4 binary; actions 17:43–05:40 UTC Mar 20
20 Mar 2026	Phase 3 — CanisterWorm deployed. 28 @EmilGroup npm packages in 60 seconds; ICP blockchain C2.	TeamPCP	npm	66+ packages by Mar 25
22 Mar 2026	Phase 3.5 — Kubernetes wiper. Iran-targeted DaemonSet wiper; SSH lateral movement; Docker API exploitation.	TeamPCP	Kubernetes	Iran and Israeli locale-targeted destruction
23 Mar 2026	Phase 4 — Checkmarx commercial AppSec pivot. ast-github-action, kics-github-action, OpenVSX plugins ast-results 2.53.0, cx-dev-assist 1.7.0.	TeamPCP	GitHub Actions, OpenVSX	2 commercial AppSec tools
24 Mar 2026	Phase 5 — LiteLLM PyPI. .pth auto-execution; Kubernetes lateral movement; bot-driven issue suppression (73 accounts, 88 comments, 102 seconds).	TeamPCP	PyPI	95M monthly downloads; 3.6M daily
25 Mar 2026	Mandiant CTO: 1,000+ SaaS environments confirmed.	—	—	1,000+ confirmed
26 Mar 2026	CISA adds CVE-2026-33634 to KEV. Federal remediation due 9 Apr 2026.	—	—	Federal mandate
27 Mar 2026	Phase 6 — Telnix PyPI. WAV steganography; C2 83[.]142[.]209[.]203:8080; ephemeral Linux exfil; Windows Startup persistence.	TeamPCP	PyPI	670K monthly downloads
31 Mar 2026	Phase 7 begins — Downstream victims. Mercor confirms LiteLLM-derived compromise. ownCloud discloses build infrastructure exposure.	TeamPCP	Post-compromise	Multiple victims
31 Mar 2026	axios npm compromised (~100M weekly downloads). Versions 1.14.1 and 0.30.4; cross-platform RAT; ~3 hours live. Attributed to Sapphire Sleet (Lazarus/DPRK) by Microsoft — not TeamPCP.	Lazarus (Sapphire Sleet)	npm	100M weekly downloads; ~3 hrs
2–3 Apr 2026	CERT-EU confirms European Commission breach via Trivy credential theft with Trivy. 91.7 GB compressed (340 GB uncompressed); 71 entities affected.	TeamPCP	AWS (Europa)	340 GB; 71 entities
8 Apr 2026	GTIG formally designates UNC6780 / SANDCLOCK.	—	—	Formal attribution
Apr 2026	Bitwarden CLI npm package compromised. Attributed to TeamPCP.	TeamPCP	npm	Password manager CLI

Date	Event	Actor	Ecosystems	Scale
19–20 Apr 2026	Context.ai / Vercel OAuth breach. Google Workspace OAuth chain; limited customer scope; Mandiant engaged; unattributed to TeamPCP.	Unattributed	Google Workspace, Vercel	"Limited" customer scope
22 Apr 2026	Second Checkmarx multi-channel attack. TeamPCP simultaneously pushed malicious images to checkmarx/kics Docker Hub repository, poisoned VS Code/OpenVSX extensions, and modified checkmarx/ast-github-action workflow — three simultaneous channels in an 83-minute window. Checkmarx confirmed and remediated the same day.	TeamPCP	Docker Hub, OpenVSX, GitHub Actions	83-min exposure window
Late Apr 2026	Shai-Hulud Gen 4 (Mini Shai-Hulud) goes active. Credential-free initial access; SLSA provenance forgery capability.	TeamPCP	npm, PyPI	Active from late April
29–30 Apr 2026	Mini Shai-Hulud debut: SAP + PyTorch Lightning + intercom-client + Ruby gems. Cross-ecosystem simultaneous strike; ~10M combined monthly installs. SAP CAP npm (Apr 29); PyTorch Lightning v2.6.2/2.6.3 PyPI (Apr 30); intercom-client downstream.	TeamPCP	npm, PyPI, RubyGems	~10M combined monthly installs
May 2026	Checkmarx Jenkins AST Scanner plugin backdoored in version 2026.5.09, distributed via the Jenkins Marketplace, and confirmed by Checkmarx per the SANS advisory (May 9; confirmed May 13).	TeamPCP	Jenkins Marketplace	Enterprise CI/CD pipelines
7 May 2026	PCPJack rival worm disclosed (SentinelLABS) and discovered April 28, 2026. Cloud worm actively evicts TeamPCP artifacts before stealing the same credentials. Targets Docker, K8s, Redis, MongoDB, RayML + 5 web app CVEs. Possible former TeamPCP affiliate; Silver C2 + Telegram C2.	PCPJack actors (possible ex-TeamPCP affiliate)	Cloud infrastructure	All systems previously compromised by TeamPCP
11 May 2026	TanStack attack (CVE-2026-45321 / GHSA-g7cv-rxg3-hmpx). Pwn request + cache poisoning + OIDC token theft — no stolen credentials required. 84 malicious artifacts / 42 @tanstack/ packages in 6 minutes. Self-propagated to 160+ namespaces; 373 total malicious versions. First SLSA Build Level 3 provenance defeat.* OpenAI: 2 employee devices compromised; signing certs rotated.	TeamPCP	npm, PyPI	373 malicious versions; 12M weekly DLs for react-router alone
12 May 2026	TeamPCP publishes the Mini Shai-Hulud source code on the public GitHub. Copycat activity begins; future attribution to TeamPCP requires corroboration beyond toolchain overlap.	TeamPCP	GitHub	Worm open-sourced
12–14 May 2026	TeamPCP + BreachForums launch \$1,000 Monero supply chain attack contest. Open-sourced Shai-Hulud tooling; scoring based on download count of compromised	TeamPCP + BreachForums operators	npm, PyPI (any ecosystem)	Ecosystem-wide

Date	Event	Actor	Ecosystems	Scale
	packages. First documented criminal contest incentivising supply chain compromise. Four confirmed copycat packages were deployed within days.			
18 May 2026	Nx Console VS Code extension trojanized (~2.2M installs) on Visual Studio Marketplace for 18 minutes. Auto-update pushed to all active users. Payload: 1Password vaults, Anthropic Claude Code configs, npm/GitHub/AWS credentials.	TeamPCP	VS Marketplace	2.2M installs; 18-min window
19–21 May 2026	GitHub internal breach + LAPSUS\$ joint data sale. GitHub employee workstation compromised via Nx Console; attacker pivoted to internal systems; ~3,800 private repos exfiltrated (GitHub confirmed as "directionally consistent"). TeamPCP and LAPSUS\$ subsequently listed stolen repositories for joint sale at \$95,000: "No ransom. If no buyer, we leak for free." Operational model: TeamPCP (initial access) → Vect (extortion capability) → LAPSUS\$ (distribution/sale). GitHub confirms breach vector on May 21.	TeamPCP + LAPSUS\$ (joint sale)	GitHub (internal)	~3,800 private repos; \$95,000 listed
May 2026 (ongoing)	Campaign active. Worm source code public; BreachForums contest driving new actors; copycat packages detected (chalk-tempalte, @deadcode09284814/axios-util, axois-utils, color-style-utils); PCPJack operating in parallel on the same victim population.	Multiple	All ecosystems	Ongoing

6. DETAILED SEQUENCE OF EVENTS BY PHASE AND ACTOR

6.1 TeamPCP Campaign — Phases 1–9

Attribution: TeamPCP (UNC6780) — HIGH CONFIDENCE across all phases (consistent RSA key pair, tpcp.tar.gz naming, tpcp-docs fallback, SANDCLOCK payload string, ICP canister C2; GTIG formal designation UNC6780 April 8, 2026). See Section 11.1 for full attribution assessment.

Phase 1 (27–28 Feb 2026): Initial Access — hackerbot-claw

AI-powered autonomous attack agent (GitHub account created 20 Feb 2026) exploited a `pull_request_target` misconfiguration in Aqua Security Trivy's `apidiff.yaml` workflow (present since October 2025; flagged by Boost Security poutine 29 Nov 2025). Exfiltrated `ORG_REPO_TOKEN` with write access to 33+ Aqua Security repositories. The agent self-describes as "powered by claude-opus-4-5", and claims support for 9 vulnerability pattern classes across 47 sub-patterns.

Phase 2 (19 Mar 2026, ~17:43 UTC): Supply Chain Injection — Trivy

Force-pushed 76 of 77 trivy-action version tags (v0.0.1–v0.34.2) and all 7 setup-trivy tags to malicious commits. Triggered Trivy v0.69.4 binary release to all distribution channels (GitHub, Docker Hub, GHCR, Amazon ECR, deb/rpm, get.trivy.dev). Malicious binary ran the legitimate Trivy scan in parallel while exfiltrating credentials, masking the compromise. Payload: process memory scraping (Runner.Worker process via `/proc/[pid]/mem`), 50+ filesystem credential paths, AES-256-CBC + RSA-4096 encrypted exfiltration to `scan.aquasecurity[.]org` (typosquat), GitHub `tpcp-docs` repository fallback. CVE-2026-33634 (CVSS 9.4); CISA KEV Mar 26.

Phase 3 (20 Mar 2026, 20:45 UTC): CanisterWorm — npm Self-Propagating Worm

28 @EmilGroup packages compromised in 60 seconds. Three-stage architecture: postinstall loader (`index.js`), Python backdoor (`service.py/pgmon`), ICP blockchain C2 (`tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io`). Dormant state: canister returns YouTube Rickroll URL. Armed state: canister points to real payload binary. Attacker rotates the entire fleet payload without republishing any packages. Self-mutation: `@teale.io/eslint-config` variant incorporates `findNpmTokens()` into postinstall hook; each compromised install autonomously propagates.

Phase 3.5 (22 Mar 2026): Kubernetes Wiper — Iran-Targeted Destruction

Same ICP canister as CanisterWorm. Four-path decision tree based on Kubernetes detection + Iran timezone/locale (Asia/Tehran, `fa_IR`):

- Kubernetes + Iran/Israel: Deploy kamikaze DaemonSet across all nodes; mount host root; execute `find /mnt/host -maxdepth 1 -not -name mnt -exec rm -rf {} +`; force-reboot all nodes
- Kubernetes + non-Iran/Israel: Deploy host-provisioner-std DaemonSet; plant persistent CanisterWorm backdoor
- Non-Kubernetes + Iran/Israel: Execute `rm -rf / --no-preserve-root` directly

- Non-Kubernetes + other: Exit silently

Note (Mini Shai-Hulud Gen 4 variant): A later variant introduces a probabilistic 1-in-6 chance of disk wipe on systems with Israeli or Iranian locale settings, rather than a deterministic wipe — reducing the wipe's detectability signature. (Source: OffSec.com, SANS diary 32994)

Later variant adds SSH lateral movement (stealing keys and spreading to a/24 subnet) and Docker API scanning (port 2375) for privileged Alpine container deployment.

Phase 4 (23 Mar 2026): Checkmarx — Commercial AppSec Tooling

Force-pushed tags in ast-github-action (before v2.3.33) and kics-github-action (before v5.0.20). Published two malicious OpenVSX plugins: ast-results-2.53.0.vsix and cx-dev-assist-1.7.0.vsix. New C2 domain: checkmarx[.]zone. Payload identical to Trivy phase; same tpcp-docs fallback pattern. Sysdig confirmed an identical RSA key, encryption scheme, and tpcp.tar.gz naming. Significance: first pivot from OSS-only to commercial AppSec tooling and IDE extension distribution.

Phase 5 (24 Mar 2026, 10:39–10:52 UTC): LiteLLM PyPI — .pth Auto-Execution

Credentials stolen from LiteLLM's CI pipeline (ran unpinned Trivy). Versions 1.82.7 and 1.82.8 have been published; the 13-minute iteration window shows active attacker tuning. Innovation: LiteLLM_init.pth auto-executes on any Python invocation, even without importing LiteLLM. If Kubernetes RBAC privileges are available, it creates DaemonSets across all cluster nodes; it enumerates all K8s secrets. Accidental discovery: .pth bug caused a fork bomb, crashing the developer's machine (~3–5 hrs window). Bot suppression: 73 accounts, 88 comments, 102 seconds. LiteLLM's strategic significance: a typical deployment simultaneously manages API keys for OpenAI, Anthropic, Azure, Google Vertex AI, and AWS Bedrock.

Phase 6 (27 Mar 2026, 03:51–04:07 UTC): Telnyx PyPI — WAV Steganography

Versions 4.87.1 and 4.87.2; malicious code in telnyx/_client.py triggers on import. Downloads ringtone.wav from C2; extracts credential harvester. Windows: drops msbuild.exe to the Startup folder (persistent). Linux/macOS: ephemeral in-memory exfil, artifacts recursively deleted (no persistence — stealth choice). Same RSA-4096 public key as LiteLLM (high-confidence same actor)—C2 HTTP (not HTTPS) — MITM vulnerable; payload validation absent.

Phase 7 (31 Mar–8 Apr 2026): Post-Telnyx Victim Expansion

Mercor (31 Mar) confirms LiteLLM-derived compromise — first downstream victim disclosure; proves LiteLLM credentials operationalized. ownCloud (31 Mar) discloses Trivy-linked build infrastructure exposure; no customer data compromised; no source code altered. Cisco (early Apr): credible reporting of development environment breach, 300+ internal repositories cloned (strongly reported; no Cisco self-disclosure). European Commission breach confirmed by CERT-EU (2–3 Apr; see Phase 8). GTIG formal designation: UNC6780 / SANDCLOCK (8 Apr).

Phase 8 (19 Mar–early Apr 2026): Downstream Cloud Exploitation

European Commission case (CERT-EU confirmed): AWS API key obtained via malicious Trivy on 19 Mar; same day used for TruffleHog --only-verified validation. Post-theft workflow: (1) validate (sts:GetCallerIdentity, Azure service principal, SaaS tokens); (2) enumerate (IAM ListUsers/ListRoles/ListAttachedUserPolicies, EC2 DescribeInstances, Lambda ListFunctions, S3/RDS/ECS); (3) persist (iam:CreateAccessKey on existing IAM user — creates second credential pair surviving original key revocation; avoids CreateUser CloudTrail events); (4) exfiltrate (91.7 GB compressed / 340 GB uncompressed / 3.7:1 ratio; ~tens of thousands email files; 42 EC clients + 29 Union entities on shared Europa platform). Detection signals: API misuse (sts:GetCallerIdentity + IAM enumeration from unexpected source IPs), account compromise (CreateAccessKey by unexpected principal), network traffic anomaly (elevated S3/VPC egress spike). Attacker resource naming: "pawn" and "massive-exfil" strings visible in CloudTrail requestParameters—no cross-account lateral movement identified despite management-level access.

Phase 9 (28 Mar–May 2026): Campaign Maturation and Escalation

The supply chain pause (28 Mar–late Apr) reflects exploitation sufficiency rather than a loss of capability. The actor accumulated sufficient credentials to sustain downstream intrusion without having to return to upstream poisoning. Campaign resumed with Mini Shai-Hulud (Gen 4) wave in late April, then TanStack (May 11), Nx Console (May 18), and GitHub breach (May 19–21).

6.2 Bitwarden CLI npm Compromise (April 2026)

Attribution: TeamPCP — STRONGLY REPORTED (Snyk, Socket, TanStack postmortem cross-references)

TeamPCP compromised the official Bitwarden CLI npm package in April 2026 as part of the ongoing Mini Shai-Hulud credential harvesting campaign. The Bitwarden CLI is notable because developers and CI/CD pipelines use it for programmatic access to password vault contents — meaning a compromised CLI installation can expose the full contents of a developer's or organization's Bitwarden vault, not merely the credentials visible in the current session. This represents a qualitative escalation over credential-harvesting attacks targeting credentials stored in files or environment variables; the CLI provides an authenticated pathway to secrets that may not exist on disk in any other form.

6.3 XPACK Attack (4 February 2026)

Attribution: Unattributed

A cluster of npm packages published by "dev.chandra_bose" employed a technique not previously documented in the wild: the HTTP 402 "Payment Required" status code as an extortion mechanism during package installation. Tracked as "XPACK ATTACK" by OpenSourceMalware.

The packages blocked installation and demanded payment of 0.1 USDC or ETH to an attacker-controlled wallet. If the developer refused to pay, the installation failed after more than five minutes of delay. The attacker collected GitHub usernames and device fingerprints from any developer who attempted installation, regardless of whether payment was made. Unlike credential-stealing attacks, this does not require post-compromise exploitation — payment creates a direct revenue stream from installation attempts, and the collected fingerprints create a developer identity database. Developers may not have recognized they encountered malware rather than a legitimate (if unusual) paywall mechanism.

6.4 axios npm Compromise (31 March 2026)

Attribution: Sapphire Sleet (Lazarus Group / BlueNoroff / DPRK) — STRONGLY REPORTED (Microsoft Threat Intelligence)

The axios npm library, with approximately 100 million weekly downloads, was compromised via a hijacked maintainer account. Two malicious versions (1.14.1 and 0.30.4) were live for approximately 3 hours before being detected and removed. The payload deployed a fully cross-platform Remote Access Trojan (RAT) targeting macOS, Windows, and Linux.

This incident is often incorrectly grouped with the TeamPCP campaign due to their temporal proximity (both occurring on the same day as Mercor's LiteLLM disclosure). Microsoft Threat Intelligence explicitly attributed the axios compromise to **Sapphire Sleet** (also tracked as BlueNoroff, a Lazarus Group sub-cluster), not to TeamPCP / UNC6780. These are distinct threat actors with different infrastructure, different payloads, and different operational objectives. The Axios compromise reflects Lazarus's willingness to target extremely high-download packages via account takeover, distinct from the fake-recruiter social-engineering vector used in the Contagious Interview campaign.

Impact scope: At ~100M weekly downloads, Axios is embedded in virtually every major JavaScript project. The three-hour exposure window, even with limited persistence, is significant given the download velocity of a package at this scale.

6.5 Lazarus Group — Contagious Interview / graphalgo Campaign (May 2025–May 2026)

Attribution: Lazarus Group (APT38, Sapphire Sleet, Famous Chollima, Shifty Corsair) — STRONGLY REPORTED

Campaign Mechanics

Fake recruiter profiles on LinkedIn, Facebook, and Reddit approach JavaScript and Python developers with job opportunities in the blockchain and cryptocurrency sector. The fake company persona "Veltrix Capital" was constructed with a registered domain, a GitHub organization, and populated repositories intended to appear as legitimate coding assessment infrastructure.

Candidates receive a coding assignment hosted on GitHub, Bitbucket, or Google Docs. The assignment code appears legitimate at first glance; the malicious behavior is not embedded directly in the interview code but is introduced through dependencies pulled from public package registries during local execution. Candidates execute the project locally to complete the assessment, triggering the malicious dependency install.

Package Mechanics

Campaign packages often establish a reputation before being weaponized. The npm package `bigmathutils` is a documented example: it accumulated over 10,000 downloads as a benign utility before attackers pushed a malicious version 1.1.0 just before February 11, 2026. The malicious update carried the same downloader payload as seen in earlier `graphalgo`-linked packages and was quickly removed, after which the package was marked deprecated. This pattern of establishing a download count before weaponizing reduces immediate suspicion and slows detection tooling that relies on reputation signals.

Campaign packages span npm, PyPI, Go modules, Rust (crates.io), and PHP Composer — coordinated cross-ecosystem targeting in parallel waves (confirmed April 2026).

Payloads: BeaverTail and InvisibleFerret

BeaverTail is a credential-stealing infostealer that targets cryptocurrency wallets (browser LocalStorage and `wallet.dat` files), SSH keys, API keys, and cloud credentials. InvisibleFerret is a persistent remote access trojan enabling follow-on intrusion. Both are multi-stage; initial package payloads serve as downloaders, fetching the final payload from the attacker's C2 infrastructure.

UNC1069 Sub-Operation

A sub-operation (UNC1069) targeted npm maintainers of high-value packages, including Fastify, Lodash, and dotenv for direct credential theft, complementing the broader fake-recruiter campaign.

6.6 Mini Shai-Hulud — SAP + PyTorch Lightning + intercom-client (29–30 April 2026)

Attribution: TeamPCP — CONFIRMED (Wiz, Socket, Aikido, The Register, OX Security, StepSecurity)

The first deployment of the Mini Shai-Hulud (Gen 4) branding was a coordinated cross-ecosystem strike across a 48-hour window:

April 29 — SAP Cloud Application Programming Model (npm):

Four official SAP JavaScript and cloud application development ecosystem npm packages were compromised between 09:55 and 12:14 UTC. Malicious preinstall scripts execute before any application code runs. Multi-stage payload: steals developer secrets, self-propagates using

stolen tokens, encrypts all stolen credentials, and exfiltrates to newly created GitHub repositories. Wiz and Socket attributed to TeamPCP.

April 30 — PyTorch Lightning (PyPI):

Versions 2.6.2 and 2.6.3 were published with malicious content. Hidden `_runtime` directory containing a downloader and an obfuscated JavaScript payload. Bun-based payload launch; heavily obfuscated JavaScript; install-time execution; credential harvesting from developer and CI/CD environments; encrypted exfiltration. The Pyannote-audio package imported Lightning as a transitive dependency, creating a second-order infection vector for downstream projects.

April 30 — intercom-client (npm):

Compromised as a downstream effect: the project's own development environment had pyannote-audio installed, which pulled in the compromised Lightning package. Intercom-client was infected as a transitive dependency chain compromise, not via direct credential theft. This is the first confirmed case in the 2026 campaign of transitive dependency infection, creating a bridge into additional package ecosystems.

Ruby gems: Same 48-hour window; attributed to the same Mini Shai-Hulud wave.

Significance: This wave introduced the Mini Shai-Hulud name and established the cross-registry simultaneous targeting pattern that would reach its peak in the TanStack attack.

6.7 TanStack Attack (11 May 2026) — CVE-2026-45321

Attribution: TeamPCP (UNC6780) — CONFIRMED (StepSecurity, Wiz, Snyk, Akamai, Socket, Palo Alto Unit 42)

Initial Access: Credential-Free (OIDC + Cache Poisoning)

An orphaned commit in a TanStack CI workflow still had OIDC trust federation configured with npm. The attacker exploited a misconfiguration in the pwn-request workflow, combined with GitHub Actions cache poisoning. A pull request from a fork triggered a workflow with write access to the base repository's cache. The attacker's code poisoned that cache and waited. The OIDC token extracted from the runner process was exchanged for authenticated npm publish access to the entire TanStack namespace. No phished maintainer credentials; no stored secret required.

Scale and Propagation

Between 19:20 and 19:26 UTC on May 11, 2026, 84 malicious package artifacts were published across 42 `@tanstack/*` packages in 6 minutes. The self-propagating worm mechanism then expanded to secondary victims: `@uipath` (66 package-version entries), `@mistralai` (npm and PyPI), `@opensearch-project/opensearch` (1.3M weekly npm downloads), `guardrails-ai` (PyPI), `@squawk`, `@tallyui`, `@beproduct`, `@draftlab`, and over 160 additional namespaces. Total: 373 malicious package-version entries across 169+ npm package names plus multiple PyPI packages.

First SLSA Build Level 3 Provenance Defeat

The malicious packages carried valid SLSA Build Level 3 provenance attestations. The attestation was genuine — generated within the legitimately configured GitHub Actions CI runner using valid OIDC tokens. The attacker who can execute in GitHub Actions can generate valid Sigstore attestations. This demonstrates that SLSA provenance is not a trust anchor when pipeline access controls are compromised. Organizations relying on provenance badges as the primary security signal are exposed.

Payload: router_init.js + Session Protocol C2

Primary payload: router_init.js (~2.3MB, heavily obfuscated using JavaScript-obfuscator pattern with string-array rotation, hex-encoded identifier lookups, control-flow flattening). The worm copies itself using the filename router_runtime.js (designed to blend into project codebases). The C2 channel is the Session decentralized messaging network — the full Session protocol stack, including the signalservice Protocol Buffers schema, is embedded directly in the 2.3 MB payload. This provides a second takedown-resistant C2 mechanism analogous to the ICP canister used in CanisterWorm.

Python Variant (Linux)

Downloads an unobfuscated modular credential stealer from C2. Exits immediately if: (a) the system uses Russian language locale settings, or (b) the system has fewer than 4 CPUs. This Russian language check mirrors the geopolitical targeting logic seen in the Iran wiper (Phase 3.5).

Exfiltration via Fake OpenTelemetry

The TanStack payload exfiltrated credentials via traffic masquerading as legitimate OpenTelemetry observability data. This is distinct from the HTTPS POST tcp.tar.gz pattern used in earlier phases and is designed to bypass egress controls that permit standard monitoring traffic.

Downstream Victims

- **OpenAI:** Two employee devices confirmed compromised; iOS, macOS, and Windows code-signing certificates rotated.
- **Mistral AI:** Official advisory MAI-2026-002; npm packages compromised via worm 22:45–01:53 UTC; PyPI compromised 00:05–03:05 UTC May 12.
- **UiPath:** 66 package-version entries affected; preinstall script (node setup.mjs) downloads Bun runtime and executes the same payload. A bug in the payload rendered the malware non-functional for UiPath and Mistral packages.

Source Code Publication (12 May 2026)

TeamPCP published the Mini Shai-Hulud worm source code to public GitHub repositories. Copycat activity immediately began. Attribution of future "Shai-Hulud-style" attacks to TeamPCP specifically should be treated with increased caution.

6.8 Nx Console VS Code Extension Trojanization (18 May 2026)

Attribution: TeamPCP — CONFIRMED (Aikido, Help Net Security, GitHub)

The Nx Console extension for Visual Studio Code (approximately 2.2 million installs; verified publisher badge; high install count conferring marketplace legitimacy) was trojanized on the Visual Studio Marketplace for **18 minutes**. The auto-update mechanism pushed the malicious version to every active user during the window.

The payload harvested: 1Password vault contents, Anthropic Claude Code configuration files, npm authentication tokens, GitHub tokens, and AWS credentials. The combination of a verified publisher badge, a high install count, and the VS Code auto-update mechanism made this an extremely high-yield attack surface. Developers do not review extension updates with the same scrutiny applied to dependency changes.

This attack demonstrated a technique that TeamPCP had itself flagged as underappreciated in their public interview (see Section 14): IDE extension governance as a critical defensive control. The Nx Console incident is the textbook proof of that assessment.

6.9 GitHub Internal Breach (19–21 May 2026)

Attribution: TeamPCP (via Nx Console → GitHub employee workstation → internal pivot) — CONFIRMED (GitHub, May 21, 2026)

A GitHub employee's developer workstation was compromised via the malicious Nx Console VS Code extension (Section 6.8). The attacker used the foothold to pivot from the workstation into GitHub's internal systems. Approximately 3,800 private internal repositories were exfiltrated (GitHub characterised this as 'directionally consistent' with their investigation), including platform source code, internal tools, and organizational resources.

The exfiltrated contents were offered for sale on underground forums with a directory listing of .tar.gz archive files provided as proof of access. GitHub publicly acknowledged the breach and confirmed the Nx Console extension as the vector on May 21, 2026.

Strategic significance: The compromise of GitHub itself — the platform that hosts the entire open-source ecosystem, manages billions of repository-level trust relationships, and serves as the distribution backbone for GitHub Actions — represents a qualitative escalation beyond the compromise of individual packages or tools. Exfiltrated GitHub internal source code enables the same offline vulnerability research risk as the Cisco breach: an attacker with the platform's internal codebase can analyze it at leisure without triggering any detection, searching for exploitable conditions in the infrastructure that every open-source project depends on.

6.10 PCPJack — Rival Worm Evicting TeamPCP (Discovered 28 April 2026)

Attribution: Possible former TeamPCP affiliate or disgruntled operator — MODERATE CONFIDENCE (SentinelLABS)

SentinelLABS first identified PCPJack on April 28, 2026, via a Kubernetes-focused VirusTotal hunting rule. Disclosed publicly on May 7, 2026 (SecurityWeek, BleepingComputer, The Register).

PCPJack is a Python-based credential-theft framework that worms its way across exposed cloud infrastructure. Its most distinguishing characteristic is that its first actions on any compromised system are to **evict processes, services, containers, and files associated with TeamPCP**, replacing them with its own tooling. The framework tracks "PCP replaced" metrics in its exfiltrated data — suggesting the author specifically designed it to operate on the same victim population as TeamPCP. SentinelOne assesses with moderate confidence that PCPJack may be operated by a former TeamPCP operator, based on tradecraft overlap with the December 2025 PCPCat phase and detailed knowledge of TeamPCP artifact paths. High-visibility TeamPCP campaigns in early 2026 reportedly led to changes in group membership, which may have triggered the shift.

Technical Mechanics

Entry point: `bootstrap.sh` (Linux shell script) that sets up the environment and fetches a full Python-based framework from attacker-controlled Amazon S3 storage (`spm-cdn-assets-dist-2026.s3.us-east-2.amazonaws.com`). Initial propagation uses five known CVEs targeting web applications:

- CVE-2025-29927: Next.js middleware authentication bypass
- CVE-2025-55182: Next.js Server Actions deserialization (React2Shell)
- CVE-2026-1357: WPVivid Backup plugin unauthenticated file upload (WordPress)
- CVE-2025-9501: W3 Total Cache PHP injection via cached `mfunc` comment (WordPress)
- CVE-2025-48703: CentOS Web Panel Filemanager `changePerm` shell injection

Inside compromised environments, PCPJack harvests credentials from cloud (AWS, GCP, Azure, DigitalOcean, Anthropic API keys), containers (Docker, Kubernetes), developer services (GitHub, npm), productivity and financial services, Discord, and messaging platforms. Establishes persistence via systemd services, cron jobs, Redis cron rewrites, or privileged containers before propagating. Lateral movement: SSH key harvesting + internal host enumeration. C2: Telegram (encrypted) + Sliver backdoor (x86_64, x86, ARM).

Strategic Significance

PCPJack represents an unprecedented dynamic in the threat landscape: a criminal-on-criminal eviction contest where two distinct actors compete for the same victim population's credentials. Defenders cannot assume that the eviction of one threat actor's artifacts means the system is clean. A system showing no TeamPCP artifacts may already be running PCPJack.

6.11 TeamPCP × BreachForums \$1,000 Supply Chain Attack Contest (14 May 2026)

Attribution: TeamPCP + BreachForums operators — STRONGLY REPORTED (SOCRadar, GBHackers, RedSide Security, Dark Web Informer)

Attribution note: Sources are threat intelligence tracking outlets and a dark-web monitoring service, not official vendor advisories or government disclosures. This meets STRONGLY REPORTED per this report's confidence tier definition, not CONFIRMED.

On or around May 14, 2026, TeamPCP and BreachForums launched a formal competition encouraging threat actors to conduct the "biggest supply chain attack" using the open-sourced Shai-Hulud tooling. The prize is \$1,000 in Monero (XMR), but the greater incentive is access, reputation, and opportunities for downstream exploitation within the cybercrime community.

Contest Structure

- Participants use open-sourced Mini Shai-Hulud tooling with their own modified C2 infrastructure.
- Scoring based on the download count of compromised packages (higher download count packages score more)
- Proof of access submitted alongside the forum identity
- Reputation points and community recognition are awarded beyond the monetary prize

Why This Matters Analytically

This is the first documented instance of a threat actor hosting a criminal supply chain attack competition — gamifying attacks against the open-source ecosystem in a manner that superficially resembles a corporate bug bounty programme but is structurally the opposite: rewarding attack success rather than responsible disclosure. It transforms supply chain compromise from a skilled operator activity into a gamified community campaign:

- Less experienced actors receive automation logic, deployment patterns, and operational examples from the worm's source code.
- The scoring model based on download counts incentivizes targeting the most widely used packages, maximizing ecosystem damage.
- Forum operators who provide infrastructure create a multi-party supply-chain attack marketplace.
- Even incomplete or partly symbolic tool releases help recruit participants by normalizing attack patterns.

Four documented copycat packages were detected by OX Security in May 2026 using open-source worm code with modified C2 infrastructure: chalk-template, @deadcode09284814/axios-util, axios-utils, and color-style-utils. These are the first confirmed contest participants, or at a minimum, direct copycats of the open-sourced tooling.

7. TECHNICAL DETAILS OF MAJOR CAMPAIGNS

7.1 TeamPCP / SANDCLOCK Technical Arsenal

Credential Harvesting (SANDCLOCK Stealer)

- Runner process memory scraping: `pgrep -f Runner.Worker`, `pgrep -f Runner.Listener`; reads `/proc/[pid]/mem`; extracts `isSecret:true` JSON patterns from runner memory
- 50+ filesystem credential paths: AWS (`~/.aws/credentials`, EC2 IMDS); GCP (service account JSON, `$GOOGLE_APPLICATION_CREDENTIALS`); Azure (service principal); SSH (`~/.ssh/`); Git (`~/.gitconfig`); GitHub (PATs, CI secrets); Kubernetes (`~/.kube/config`, service account tokens); Docker (`~/.docker/config.json`); npm (`~/.npmrc`, `NPM_TOKEN`); PyPI (`~/.pypirc`, `TWINE_PASSWORD`); Terraform state files; `.env` files; shell history; cryptocurrency wallets
- Cloud metadata: AWS IMDSv1 (Checkmarx phase) and IMDSv2 (LiteLLM phase)
- Encryption: AES-256-CBC + RSA-4096 OAEP hybrid; `tpcp.tar.gz` archive; X-Filename: `tpcp.tar.gz` exfiltration header

Mini Shai-Hulud Enhancements (Gen 4)

- Credential-free initial access: GitHub Actions cache poisoning + OIDC token extraction + npm publish exchange.
- Session Protocol C2: full Session Protocol Buffers schema embedded in payload; decentralized messaging network used for all C2 traffic (takedown-resistant; second novel blockchain-adjacent C2 after ICP canister)
- Exfiltration masquerade: fake OpenTelemetry observability traffic; bypasses egress controls, permitting monitoring traffic.
- Russian locale + CPU count check: Python variant exits on Russian locale or `<4` CPUs
- Sandbox evasion retained: 5-minute initial sleep

Persistence Mechanisms

- systemd user services: `sysmon.py`, `pgmon.service`, `internal-monitor`, `service`, `pgmonitor`, `audiomon` (earlier phases)
- Gen 4 additions: `gh-token-monitor.service` (Linux), `com.user.gh-token-monitor.plist` (macOS LaunchAgent)
- Python `.pth` file auto-execution (LiteLLM phase)
- Kubernetes DaemonSets: `host-provisioner-iran`, `host-provisioner-std`
- AWS IAM: `iam:CreateAccessKey` on existing users (second credential pair; survives original key revocation)
- Windows Startup folder: `msbuild.exe` (Telnyx phase)

C2 Infrastructure (Multi-Generation)

- ICP blockchain canister: `tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io` (CanisterWorm phases)

- Session decentralized messaging: seed1/2/3.getsession.org, filev2.getsession.org (TanStack phase)
- Typosquat domains: scan.aquasecurity[.]org, checkmarx[.]zone, models.LiteLLM[.]cloud
- IP infrastructure: 83.142.209.0/24 subnet (cross-campaign)
- Cloudflare Quick Tunnels: relay infrastructure
- GitHub repository creation: tpcp-docs / docs-tpcp (credential exfiltration fallback)

7.2 Lazarus Technical Arsenal

Fake Recruiter Infrastructure

Veltrix Capital domain registration; GitHub organization with populated repositories; coding assessment materials on GitHub/Bitbucket/Google Docs. Sustained personas across LinkedIn, Facebook, and Reddit. UNC1069 sub-operation directly targeted npm maintainers of high-value packages.

Payload Families

BeaverTail (infostealer): cryptocurrency wallets, browser autofill, SSH keys, cloud credentials. InvisibleFerret (RAT): persistent backdoor; remote access for follow-on intrusion. Multi-stage delivery: packages serve as downloaders; the final payload is fetched from the attacker's C2. Obfuscation: hex encoding, base64 layers, dynamic code generation, generative AI-assisted code generation.

axios Phase (Mar 31, 2026 — Sapphire Sleet)

Maintainer account takeover. Versions 1.14.1, 0.30.4. Cross-platform RAT. ~3 hours live. Distinct from CanisterWorm in infrastructure, payload, and operational fingerprint.

8. CHARTS, DIAGRAMS, AND INDICATORS

8.1 Worm Generation Evolution

Gen 1 – Shai-Hulud (Sep 2025)

Initial access: Stolen npm token (seed)
Mechanism: Stolen token → npm search → inject → publish
C2: GitHub marker repos
Novel: First self-propagating npm worm

Gen 2 – SHA1-Hulud / Shai-Hulud 2.0 (Nov 2025)

Escalations: Bun runtime obfuscation; preinstall execution;
dead-man's-switch wiper; botnet secret-sharing;
self-hosted runner registration (SHA1HULUD)
Scale: 700+ packages; 33,185 secrets

Gen 3 – SANDWORM_MODE (Early 2026)

Escalations: Adaptive CI/CD enumeration before propagation;
evolved to address prior detection and takedown techniques

Gen 4 – Mini Shai-Hulud (Late Apr 2026–present)

Escalations: Credential-free initial access (OIDC + cache poisoning);
First SLSA Build Level 3 provenance forgery;
Session Protocol decentralized C2;
fake OpenTelemetry exfiltration masquerade;
Russian locale + CPU count sandbox evasion
Peak: TanStack 373 malicious versions; GitHub breach (May 2026)
Status: Source code published publicly on May 12, 2026

8.2 Threat Actor Operational Model Comparison

Dimension	TeamPCP (UNC6780)	Lazarus (DPRK)	PCPJACK	Shai-Hulud lineage	Ultralytics
Motivation	Financial	State-sponsored	Financial (credential theft)	Financial	Unknown
Initial Access	GitHub Actions token + pull_request_target; OIDC/cache (Gen 4); VS Code extension	Fake recruiter social engineering	CVE exploitation (5 web app CVEs); exposed Docker/K8s APIs	Stolen npm tokens; OIDC (Gen 4)	GitHub Actions script injection
Ecosystem Focus	npm, PyPI, Docker, Actions, OpenVSX, VS Marketplace, GitHub platform	npm, PyPI, Go, Rust, PHP	Cloud infra (Docker, K8s, Redis, MongoDB)	npm (primary)	PyPI
Payload	SANDCLOCK credential stealer; CanisterWorm worm; Iran/Israel wiper; RAT (via IDE ext)	BeaverTail infostealer; InvisibleFerret RAT	Python credential theft framework; evicts TeamPCP artifacts	Self-propagating worm	XMRig cryptominer
C2	ICP blockchain + Session Protocol + typosquat domains + GitHub repos	Traditional HTTPS C2	Telegram (encrypted) + Sliver implant	GitHub marker repos; Session Protocol (Gen 4)	Direct binary
Persistence	systemd; .pth; K8s DaemonSets; AWS IAM key; Windows Startup; LaunchAgent	RAT backdoor	systemd; cron; Redis cron rewrite; privileged containers	SHA1HULUD self-hosted runners	Binary execution
AI/Automation	hackerbot-claw (AI initial access scanning)	AI-assisted code generation (package payloads)	Not documented	Not documented	Not documented
Scale	66+ npm; 4+ PyPI; 2 Docker; 2 OpenVSX; 1 VS Marketplace; GitHub internal	1,700+ packages across 5 ecosystems; axios (100M DLs)	All TeamPCP-compromised systems (victim population theft)	700+ npm (Nov wave); 373 versions (TanStack)	4 PyPI versions
Highest-Impact Victim	GitHub (internal breach); European Commission (340 GB)	axios (100M weekly DLs); developer population via recruiting	TeamPCPs' own victim population (credential competition)	Zapier, PostHog, Postman (Shai-Hulud 2.0)	Ultralytics itself

9. INDICATORS OF COMPROMISE (IOCs)

9.1 TeamPCP / CanisterWorm / SANDCLOCK IOCs (Phases 1–6)

Network Indicators

- C2 Domains (defanged): scan.aquasecurity[.]org (typosquat), checkmarx[.]zone, models.LiteLLM[.]cloud, 83[.]142[.]209[.]203:8080 (Telnyx C2)
- IP Addresses: 45[.]148[.]10[.]212, 83[.]142[.]209[.]203, 83[.]142[.]209[.]111, 46[.]151[.]182[.]203
- IP subnet: 83[.]142[.]209[.]0/24 (linked to multiple TeamPCP operations)
- ICP Canister (CanisterWorm C2): tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0[.]io
- Cloudflare Tunnels: plug-tab-protective-relay.trycloudflare[.]com, souls-entire-defined-routes.trycloudflare[.]com, investigation-launches-hearings-copying.trycloudflare[.]com, championships-peoples-point-cassette.trycloudflare[.]com
- Precursor (hackerbot-claw): hackmoltrepeat[.]com/molt, recv.hackmoltrepeat[.]com

File / Service Artifacts (Phases 1–6)

- tcp.tar.gz (encrypted exfiltration archive; X-Filename: tcp.tar.gz HTTP header — universal campaign signature)
- tcp-docs / docs-tcp repository pattern (GitHub fallback exfiltration marker)
- ~/.config/sysmon/sysmon.py, ~/.config/systemd/user/sysmon.service (Trivy binary persistence)
- ~/.local/share/pgmon/service.py, ~/.config/systemd/user/pgmon.service (CanisterWorm persistence)
- /tmp/pglog, /tmp/pg_state (CanisterWorm payload staging)
- ~/.config/audiomon/audiomon.py, ~/.config/systemd/user/audiomon.service (Telnyx variant)
- LiteLLM_init.pth (Python auto-execution on any Python invocation)
- %APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\msbuild.exe, msbuild.exe.lock (Windows Telnyx persistence)

Package Versions (Phases 1–6 — Compromised)

- aquasecurity/trivy binary: v0.69.4, v0.69.5, v0.69.6
- Docker Hub trivy digests:
 - sha256:27f446230c60bbf0b70e008db798bd4f33b7826f9f76f756606f5417100beef3 (0.69.4),
 - sha256:5aaa1d7cfa9ca4649d6ffad165435c519dc836fa6e21b729a2174ad10b057d2b (0.69.5),
 - sha256:425cd3e1a2846ac73944e891250377d2b03653e6f028833e30fc00c1abbc6d33 (0.69.6)
- trivy-action tags: v0.0.1–v0.34.2 (force-pushed; all 76 tags malicious)
- setup-trivy: all 7 tags before v0.2.6 rebuild
- Checkmarx ast-github-action: before v2.3.33; kics-github-action: before v5.0.20

- OpenVSX: ast-results-2.53.0.vsix, cx-dev-assist-1.7.0.vsix
- LiteLLM PyPI: 1.82.7, 1.82.8
- telnyx PyPI: 4.87.1, 4.87.2
- npm namespaces (66+ packages): @EmilGroup, @opengov, @teale.io, @airtm, @pypestream

Hashes (Sample)

- Trivy v0.69.4 Linux-64bit:
822dd269ec10459572dfaaefe163dae693c344249a0161953f0d5cdd110bd2a0
- Trivy v0.69.4 Linux-ARM64:
e64e152afe2c722d750f10259626f357cdea40420c5eedae37969fbf13abbecf
- Trivy v0.69.4 macOS-ARM64:
6328a34b26a63423b555a61f89a6a0525a534e9c88584c815d937910f1ddd538
- Trivy v0.69.4 Windows-64bit:
0880819ef821cff918960a39c1c1aada55a5593c61c608ea9215da858a86e349
- Malicious entrypoint.sh (17,592 bytes):
18a24f83e807479438dcab7a1804c51a00dafc1d526698a66e0640d1e5dd671a
- Legitimate entrypoint.sh (2,855 bytes):
07500e81693c06ef7ac6bf210cff9c882bcc11db5f16b5bded161218353ba4da
- telnyx==4.87.1:
7321caa303fe96ded0492c747d2f353c4f7d17185656fe292ab0a59e2bd0b8d9
- telnyx==4.87.2:
cd08115806662469bbedec4b03f8427b97c8a4b3bc1442dc18b72b4e19395fe3
- CanisterWorm index.js Wave 1:
e9b1e069efc778c1e77fb3f5fcc3bd3580bbc810604cbf4347897ddb4b8c163b
- CanisterWorm index.js Wave 2:
61ff00a81b19624adaad425b9129ba2f312f4ab76fb5ddc2c628a5037d31a4ba
- CanisterWorm deploy.js Wave 1:
f398f06eefcd3558c38820a397e3193856e4e6e7c67f81ecc8e533275284b152
- CanisterWorm deploy.js Wave 2:
7df6cef7ab9aae2ea08f2f872f6456b5d51d896ddda907a238cd6668ccdc4bb7

Process / Behavioral Indicators (Phases 1–6)

- pgrep -f Runner.Worker; pgrep -f Runner.Listener (credential stealer targeting GitHub Actions runner process)
- /proc/[pid]/mem scraping (memory-based secret extraction)
- Outbound HTTP POST with X-Filename: tpcp.tar.gz (exfiltration signature)
- ICP canister polling to icp0.io (C2 beaconing)
- Kubernetes DaemonSet creation: hostNetwork: true, hostPID: true, privileged: true
- kubectl get secrets --all-namespaces -o json (K8s secret enumeration)
- AWS iam:CreateAccessKey on existing IAM user (persistence)
- sts:GetCallerIdentity (credential validation)
- CloudTrail requestParameters containing "pawn" or "massive-exfil" strings

9.2 Mini Shai-Hulud / TanStack IOCs (CVE-2026-45321) — Gen 4

Network Indicators

- C2 domains (defanged): git-tanstack[.]com (primary C2), api.masscan[.]cloud (alternate C2)
- Session Protocol decentralized C2: seed1.getsession.org, seed2.getsession.org, seed3.getsession.org (seed nodes), filev2.getsession.org (file server)
- IP: 83[.]142[.]209[.]194 (Python variant C2 — note: same /24 subnet as earlier TeamPCP phases)
- Block at DNS/proxy level: git-tanstack.com, *.getsession.org

File Artifacts

- router_init.js (~2.3 MB, javascript-obfuscator pattern: string-array rotation, hex-encoded lookups, control-flow flattening)
- router_runtime.js (worm self-copy filename; designed to blend into project codebases)
- .claude/router_runtime.js, .claude/settings.json, .claude/setup.mjs (Claude Code config directory injection)
- .vscode/setup.mjs, .vscode/tasks.json (VS Code config directory injection)

Persistence Artifacts

- macOS: ~/Library/LaunchAgents/com.user.gh-token-monitor.plist
- Linux: ~/.config/systemd/user/gh-token-monitor.service

Git / Repository Markers

- Git dependency marker:
github:tanstack/router#79ac49eedf774dd4b0cfa308722bc463cfe5885c
- Campaign strings: commits containing `thebeautifulmarchoftime` — if it also contains `the beautiful sandsoftime`, this indicates an active or historical C2 domain beacon
- GitHub repo search: any repo with router_init.js or router_runtime.js in unexpected paths

Affected Package Namespaces (Primary)

- @tanstack/* (42 packages; versions published 19:20–19:26 UTC May 11, 2026)
- @uipath/* (66 package-version entries)
- @mistralai/* (npm + PyPI mistralai==2.4.6)
- @opensearch-project/opensearch
- guardrails-ai (PyPI)
- @squawk/, @tallyui/, @beproduct/, @draftlab/ and 160+ additional namespaces
- Total: 373 malicious package-version entries

CVEs / GHSAs

- CVE-2026-45321 / GHSA-g7cv-rxg3-hmpx (TanStack)
- CVE-2026-33634 / GHSA-69fq-xp46-6x23 (Trivy)

- GO-2026-4919 (Trivy Go module)
- CVE-2025-30066 (tj-actions); CVE-2025-30154 (reviewdog)

Detection Tool

- <https://github.com/ry-allan/tanstack-compromise-checker> (scans node_modules, lockfiles, persistence hooks, GitHub workflows, git history, C2 domains, and AI tool configs)

9.3 Lazarus / Contagious Interview IOCs

Package Indicators

- graphalgo (first npm package in campaign; campaign named after this)
- bigmathutils — benign until malicious v1.1.0 (~Feb 11, 2026); 10,000+ downloads before weaponization
- "big" prefix package series (second wave from December 2025)
- Packages with initial benign versions followed by a malicious patch/minor update after reputation is established

Behavioral Indicators

- Cryptocurrency wallet file access (LocalStorage, wallet.dat)
- BeaverTail multi-stage download pattern
- Fake "Veltrix Capital" GitHub org/repository structure

9.4 Shai-Hulud Gen 1/2 IOCs

Network / Repository Markers

- Shai-Hulud marker repository creation pattern (GitHub credential exfiltration fallback)
- Self-hosted runner registration: hostnames containing "SHA1HULUD"
- Patient-zero package: @ctrl/tinycolor v4.1.1 (Gen 1 initial compromise)

File Indicators

- deploy.js Wave 1:
f398f06eefcd3558c38820a397e3193856e4e6e7c67f81ecc8e533275284b152
- deploy.js Wave 2:
7df6cef7ab9aae2ea08f2f872f6456b5d51d896ddda907a238cd6668ccdc4bb7
- deploy.js Wave 3+:
5e2ba7c4c53fa6e0cef58011acdd50682cf83fb7b989712d2fcf1b5173bad956

9.5 Context.ai / Vercel (April 2026) — Unattributed

Context: Google Workspace OAuth compromise chain; Vercel CEO-confirmed breach; unattributed to any named threat actor in this report. NOT a Lazarus Group IOC.

OAuth Application Indicator

- Google Workspace OAuth client ID (immediately audit and revoke if present):
[110671459871-30f1spbu0hptbs60cb4vsmv79i7bbvqj.apps.googleusercontent.com](https://console.developers.google.com/apis/credentials/110671459871-30f1spbu0hptbs60cb4vsmv79i7bbvqj.apps.googleusercontent.com)
- Check via: Google Workspace Admin Console → Reporting → Audit and investigation → OAuth log events.

9.6 PCPJack IOCs

Attribution: Possible former TeamPCP affiliate — MODERATE CONFIDENCE (SentinelLABS)

Discovered: April 28, 2026 | **Disclosed:** May 7, 2026

Entry Point / Staging Infrastructure

- Entry script: [bootstrap.sh](#) (Linux shell script; sets up environment and fetches Python framework)
- S3 staging bucket: [spm-cdn-assets-dist-2026.s3.us-east-2.amazonaws.com](#) (attacker-controlled; Python framework download source)
- C2: Telegram (encrypted channel); Sliver C2 implant (x86_64, x86, ARM variants)

Exploitation CVEs (Initial Access)

- CVE-2025-29927: Next.js middleware authentication bypass
- CVE-2025-55182: Next.js Server Actions deserialization (React2Shell)
- CVE-2026-1357: WPVivid Backup plugin unauthenticated file upload (WordPress)
- CVE-2025-9501: W3 Total Cache PHP injection via cached mfunc comment (WordPress)
- CVE-2025-48703: CentOS Web Panel Filemanager changePerm shell injection

Behavioral Indicators

- Evicts TeamPCP artifacts on first execution: kills TeamPCP processes, removes TeamPCP service files, and deletes TeamPCP persistence mechanisms.
- Tracks "PCP replaced" metric in exfiltrated data (key hunting indicator — look for this string in network egress or process output)
- Targets the same credential paths as TeamPCP: AWS, GCP, Azure, Digital Ocean, Anthropic API keys, Docker, Kubernetes, GitHub, npm, Discord, and financial services
- Persistence: systemd services, cron jobs, Redis cron rewrites, privileged containers
- Lateral movement: SSH key harvesting + internal host enumeration + Kubernetes/Docker API scanning

Defensive Note: A system with no TeamPCP artifacts is NOT confirmed clean. PCPJack actively removes TeamPCP artifacts before deploying its own tooling. Incident responders must hunt for PCPJack indicators independently of the presence of TeamPCP artifacts.

10. MITRE ATT&CK MAPPING

10.1 TeamPCP / UNC6780 (Phases 1–9 including Mini Shai-Hulud)

Tactic	Technique ID	Technique	Evidence
Reconnaissance	T1596	Search Open Technical Databases	hackerbot-claw systematic repo scanning for pull_request_target misconfigs
Resource Development	T1583.001	Acquire Infrastructure: Domains	Typosquats: scan.aquasecurity[.]org, checkmarx[.]zone, models.LiteLLM[.]cloud, git-tanstack[.]com
Resource Development	T1583.006	Acquire Infrastructure: Web Services	ICP blockchain canister; Session Protocol decentralized messaging; Cloudflare Tunnels
Resource Development	T1587.001	Develop Capabilities: Malware	SANDCLOCK stealer; CanisterWorm; Mini Shai-Hulud; Iran wiper; WAV stego payloads
Initial Access	T1195.002	Supply Chain: Software Supply Chain	Tag poisoning (trivy-action 76 tags, setup-trivy, Checkmarx); backdoored binaries; OIDC-credential-free (TanStack)
Initial Access	T1195.001	Supply Chain: Software Dependencies	CanisterWorm and Mini Shai-Hulud are infecting consumed npm/PyPI packages
Initial Access	T1078.004	Valid Accounts: Cloud Accounts	Compromised aqua-bot, cx-plugins-releases service accounts
Initial Access	T1199	Trusted Relationship	Trivy, Checkmarx, LiteLLM as trusted CI/CD components; Nx Console as trusted VS Code extension
Execution	T1059.004	Command & Scripting: Unix Shell	Malicious entrypoint.sh; kamikaze.sh wiper
Execution	T1059.006	Command & Scripting: Python	sysmon.py backdoor; pgmon.py C2 poller; audiomon credential harvester; SANDCLOCK Python stager
Execution	T1204.002	User Execution: Malicious File	npm postinstall/preinstall hooks; .pth auto-execution; VS Code extension install
Persistence	T1543.002	Systemd Service	sysmon.service; pgmon.service; internal-monitor; pgmonitor; audiomon; gh-token-monitor.service
Persistence	T1543.001	Launch Agent (macOS)	com.user.gh-token-monitor.plist (macOS LaunchAgent — Mini Shai-Hulud Gen 4; T1543.001 per ATT&CK v15)
Persistence	T1098.001	Account Manipulation: Additional Cloud Credentials	iam:CreateAccessKey on existing IAM users (European Commission case confirmed)
Defence Evasion	T1036.005	Masquerading	PostgreSQL-themed naming (pgmon, pglog); sysmon mimics system monitor; msbuild.exe mimics Microsoft Build; router_runtime.js blends into project

Tactic	Technique ID	Technique	Evidence
Defence Evasion	T1070.004	Indicator Removal: File Deletion	Telnix Linux variant recursively deletes artifacts; artifact cleanup post-exfil
Defence Evasion	T1480.001	Execution Guardrails	5-min sleep (sandbox evasion); timezone/locale checks (Iran targeting); Russian locale + CPU count check (Mini Shai-Hulud)
Defence Evasion	T1027.003	Obfuscated Files: Steganography	WAV steganography (ringtone.wav, hangup.wav, bg_kube.wav)
Defence Evasion	T1036	Impersonation	Imposter commits spoofing rauchg (Vercel CEO), DmitryLewen (Trivy maintainer)
Credential Access	T1003.007	OS Credential Dumping: /proc	/proc/[pid]/mem scraping targeting Runner.Worker
Credential Access	T1552.001	Unsecured Credentials: Files	50+ filesystem credential paths; 1Password vault harvesting (Nx Console)
Credential Access	T1552.004	Unsecured Credentials: Private Keys	SSH, TLS/SSL key harvesting
Credential Access	T1552.005	Steal Cloud Instance Metadata	AWS IMDS querying
Credential Access	T1528	Steal Application Access Token	OIDC token theft and npm publish exchange (TanStack — credential-free)
Collection	T1560.003	Archive via Custom Method	AES-256-CBC + RSA-4096 hybrid → tcp.tar.gz
Collection	T1119	Automated Collection	CanisterWorm findNpmTokens() + deploy.js autonomous harvesting and republishing
Exfiltration	T1041	Exfiltration Over C2 Channel	HTTPS POST to typosquat domains; Session Protocol; 340 GB European Commission exfil
Exfiltration	T1537	Transfer Data to Cloud Account	tcp-docs GitHub repos as fallback exfiltration
Exfiltration / C2	T1071.001	Application Layer Protocol: Web	HTTPS POST to typosquat C2 domains; fake OpenTelemetry observability traffic masquerading exfiltration as legitimate CI/CD telemetry to bypass egress controls (Mini Shai-Hulud Gen 4)
C2	T1090.003	Multi-hop Proxy	Cloudflare Tunnel relay infrastructure
C2	T1102.002	Web Service: Bidirectional Communication	ICP blockchain canister via icp0.io (CanisterWorm — novel non-standard C2 channel); Session Protocol decentralized messaging (Mini Shai-Hulud — takedown-resistant)
Impact	T1657	Financial Theft	Credential-driven extortion; underground credential sales
Impact	T1561.002	Disk Wipe	Iran and Israeli locale-targeted wiper: kamikaze.sh DaemonSet (K8s); rm -rf / on standalone hosts; Gen 4 variant uses probabilistic 1-in-6 wipe

Tactic	Technique ID	Technique	Evidence
Impact	T1491.002	Defacement	44 Aqua repos renamed in 79-second burst

10.2 Lazarus (Contagious Interview) ATT&CK

Tactic	Technique ID	Technique	Evidence
Reconnaissance	T1566.003	Phishing: Spearphishing via Service	LinkedIn/Facebook/Reddit fake recruiter personas; Veltrix Capital brand; job postings on Reddit
Initial Access	T1199	Trusted Relationship	Trust in the legitimate job-interview process; fake coding assignment infrastructure
Initial Access	T1195.001	Supply Chain: Software Dependencies	Malicious npm/PyPI/Go/Rust/PHP packages embedded in coding assignment repositories
Execution	T1204.002	User Execution: Malicious File	Candidate executes coding assignment locally; npm/pip install triggers payload
Credential Access	T1552.001	Unsecured Credentials: Files	Cryptocurrency wallets, SSH keys, API keys, cloud credentials from developer machines
Persistence	T1543.002	Systemd Service	InvisibleFerret RAT establishes a persistent backdoor via service installation
C2	T1071.001	Application Layer Protocol: Web	BeaverTail/InvisibleFerret C2 via HTTPS to attacker-controlled endpoints
Impact	T1657	Financial Theft	Cryptocurrency wallet theft; credential monetization; DPRK revenue generation

10.3 PCPJack ATT&CK Coverage

Tactic	Technique ID	Technique	Evidence
Reconnaissance	T1595.002	Active Scanning: Vulnerability Scanning	Scans for exposed Docker APIs, Kubernetes, Redis, MongoDB, and RayML services
Initial Access	T1190	Exploit Public-Facing Application	CVE-2025-29927, CVE-2025-55182, CVE-2026-1357, CVE-2025-9501, CVE-2025-48703
Defence Evasion	T1070.004	Indicator Removal: File Deletion	Actively removes TeamPCP artifacts, processes, and service files before deploying own tooling
Credential Access	T1552.001	Unsecured Credentials: Files	Sweeps the same credential paths as TeamPCP: AWS, GCP, Azure, GitHub, npm, Digital Ocean
Credential Access	T1552.004	Unsecured Credentials: Private Keys	SSH key harvesting for lateral movement
Persistence	T1543.002	Systemd Service	Installs persistence via systemd services, cron jobs, and Redis cron rewrites
Persistence	T1610	Deploy Container	Persistence via privileged Docker containers with host root mounted, enabling container escape and host-level persistence
Lateral Movement	T1021.004	Remote Services: SSH	SSH lateral movement using harvested keys
C2	T1071.001	Application Layer Protocol: Web	Telegram encrypted C2 channel
C2	T1572	Protocol Tunneling	Sliver C2 implant (x86_64, x86, ARM variants)
Collection	T1119	Automated Collection	Programmatic sweeping of cloud, developer, financial, and messaging service credentials
Exfiltration	T1041	Exfiltration Over C2 Channel	Credential exfiltration via a Telegram encrypted channel

11. ATTRIBUTION ASSESSMENT

11.1 TeamPCP / UNC6780 / SANDCLOCK

Confidence: HIGH Consistent technical fingerprint across all phases: RSA key pair, tpcp.tar.gz naming, tpcp-docs fallback, TeamPCP Cloud stealer string, ICP canister C2, typosquat domain pattern. Formal designation: GTIG UNC6780 (8 April 2026); SANDCLOCK malware family. Primary source attribution: Snyk, Wiz, Datadog, Endor Labs, StepSecurity, Sysdig, Microsoft, Akamai, Palo Alto Unit 42.

Attribution of Full Shai-Hulud Lineage to TeamPCP: STRONGLY REPORTED Multiple credible vendors (Tenable FAQ, Security Boulevard FAQ, Akamai) retrospectively attribute the full Shai-Hulud worm lineage — including Gen 1 (September 2025) and Gen 2 (November 2025) — to TeamPCP/UNC6780. This is a material update from earlier reports that treated the September–November 2025 waves as unattributed. The earlier non-attribution was a defensible conservative position; the current weight of vendor evidence supports TeamPCP authorship of the full lineage.

A critical temporal caveat applies: Gen 1 (Shai-Hulud) appeared in September 2025, but TeamPCP was not formally designated as a distinct intrusion set (UNC6780) by Google Threat Intelligence Group until April 8, 2026 — a seven-month gap. All vendor attribution of the September–November 2025 waves to TeamPCP is therefore retrospective, applied backward from the April 2026 formal designation rather than contemporaneous with the original incidents. This does not invalidate the attribution — shared infrastructure, consistent RSA key pair, identical tpcp.tar.gz exfiltration pattern, and ICP canister C2 linkages are substantive technical connectors — but it means the confidence rests on post-hoc correlation rather than real-time tracking. Analysts building prosecution-grade or intelligence-product-grade assessments should note this distinction. Maintained as STRONGLY REPORTED pending definitive primary-source government confirmation.

Vect Ransomware Partnership: STRONGLY REPORTED (*upgraded from UNVERIFIED*) Akamai explicitly reported Vect's announced partnership with TeamPCP in their Mini Shai-Hulud analysis. Akamai is a credible primary security vendor; this is not an extortion-side or forum claim. Upgrade from UNVERIFIED to STRONGLY REPORTED. No US/allied government confirmation. Operationally significant: credential theft may now be followed by ransomware deployment through Vect, changing the expected follow-on incident pattern.

LAPSUS\$ Collaboration on GitHub Data Sale: CONFIRMED Following the GitHub internal breach (May 19–21, 2026), LAPSUS\$ and TeamPCP jointly listed the stolen GitHub repositories for sale at

95,000 on underground forums, with an accompanying statement: "Everything for the main platform is here. No ransom, we do not care about extorting GitHub. If no buyer is found, we leak for free." (Source: The Hacker News, via Dark Web Informer screenshots.) The operational model reported by multiple vendors: TeamPCP provides initial access via supply chain compromise; Vect handles encryption and extortion; LAPSUS\$ 95,000 on underground forums, with an accompanying statement: "Everything for the main platform is there. No ransom, we do not care about extorting GitHub. If no buyer is found, we leak for free." (Source: The Hacker News, via Dark Web Informer screenshots.) The operational model reported by multiple vendors: TeamPCP provides initial access via supply chain compromise; Vect handles encryption and extortion; LAPSUS\$

95,000 on underground forums, with an accompanying statement: "Everything for the main platform is here. No ransom, we do not care about extorting GitHub. If no buyer is found, we leak for free." (Source: The Hacker News, via Dark Web Informers screenshots.) The operational model reported by multiple vendors: TeamPCP provides initial access via supply chain compromise; Vect handles encryption and extortion; LAPSUS handles distribution and sale of stolen data. This upgrades LAPSUS\$ collaboration from UNVERIFIED to CONFIRMED for the specific activity of jointly monetizing the GitHub breach data. Broader operational merger across earlier phases of the campaign is not confirmed and should not be inferred from this single joint sale.

Geographic Assessment: Timestamp analysis (SANS) shows activity clusters UTC-4 to UTC-9; likely Western hemisphere 2–4 person team. No Russian/Chinese/Farsi language indicators.

11.2 Lazarus Group / DPRK (Contagious Interview / graphalgo / Sapphire Sleet)

Confidence: STRONGLY REPORTED

Attribution based on: ReversingLabs Contagious Interview campaign formal report; Sapphire Sleet attribution for Axios by Microsoft Threat Intelligence; consistent payload families (BeaverTail, InvisibleFerret); DPRK state-sponsored assessment from multiple vendors. No US government official confirmation as of May 2026.

Key Attribution Notes:

- axios compromise: explicitly attributed to Sapphire Sleet (Lazarus), NOT TeamPCP; organizations must not conflate these campaigns
- graphalgo campaign: ReversingLabs formal attribution to Lazarus; active since May 2025
- UNC1069 sub-operation: specifically targets package maintainers

11.3 Shai-Hulud (Worm Lineage — Gen 1 and Gen 2)

Revised Assessment: STRONGLY REPORTED attribution to TeamPCP/UNC6780 based on multiple security vendor retrospective analyses. Earlier treatment as "unattributed with possible UNC6426 link" should be updated. The credential-pool link from s1ngularity to Shai-Hulud remains an unconfirmed hypothesis.

11.4 Context.ai / Vercel (April 2026)

Confidence: NOT ATTRIBUTED TO TeamPCP — CEO-level vendor confirmation of breach

The Vercel CEO confirmed a breach via the Context.ai OAuth chain. Mandiant is engaged in response; no GTIG attribution to UNC6780 as of May 2026. ShinyHunters BreachForums claim remains in the CLAIMED tier. Distinct technical fingerprint from the SANDCLOCK playbook. Maintained as a separate analysis pending primary-source attribution from Mandiant or GTIG.

11.5 XPACK Attack / Ultralytics / tj-actions

All remain **UNATTRIBUTED**. No threat actor claims; no primary source attribution.

12. SUPPLY CHAIN ATTACK LINEAGE AND EVOLUTION

12.1 Technique Evolution by Campaign

Campaign	Initial Access	Propagation	Persistence	Payload	Exfiltration	Novel Elements
tj-actions (Mar 2025)	Stolen GitHub PAT; 4-hop chain	Limited (slow cascade)	Logs (secrets printed)	Memory dump to logs	GitHub Actions logs	Mutable tag rewrite at scale
s1ngularity (Aug 2025)	pull_request_target exploit	Single burst; no worm	Shell startup modification	Credential harvester + AI CLI tool abuse	Attacker C2	First AI CLI tool weaponization
Shai-Hulud Gen 1 (Sep 2025)	Stolen npm token (seed)	Autonomous npm publish	Self-hosted runner (SHA1HULUD)	Token stealer	GitHub marker repos	First self-propagating npm worm
Shai-Hulud Gen 2 (Nov 2025)	Same as Gen 1	Preinstall + Bun obfuscation; botnet token-sharing	Runners + botnet nodes	Escalated (33K secrets)	GitHub + botnet	Wiper dead-man's-switch; credential botnet
Shai-Hulud Gen 3 (Early 2026)	Same	Adaptive (CI/CD enumeration before propagating)	Same	Same	Same	Adaptive CI/CD profiling
TeamPCP (Feb–Mar 2026)	AI-powered hackerbot-claw + pull_request_target	Cross-ecosystem chaining (5 ecosystems, 9 days)	systemd + .pth + K8s DaemonSets + AWS IAM key creation	SANDCLOCK (50+ paths + memory scraping)	Blockchain canister + typosquat domains + GitHub	ICP blockchain C2; security tooling targeting; Iran + Israel wiper
Mini Shai-Hulud Gen 4 (Apr–May 2026)	Credential-free: OIDC + cache poisoning (no stolen credentials)	Self-propagating via OIDC token re-exchange	LaunchAgent (macOS); gh-token-monitor.service (Linux)	SANDCLOCK + Session C2 + OTel masquerade	Session Protocol decentralized messaging; fake OTel	First SLSA L3 provenance defeat; Session Protocol C2; credential-free initial access; source code published publicly

12.2 Interval Compression

Period	Major Events	Interval
2020–2021	SolarWinds → Codecov	~4 months
2021–2025	Codecov → tj-actions	~4 years
Mar–Nov 2025	tj-actions → s1ngularity → Shai-Hulud Gen 1 → Gen 2	8 months total
Dec 2025–May 2026	PCPcat → TeamPCP Phases 1–6 → Mini Shai-Hulud → TanStack → Nx Console → GitHub	6 months; phases in days

Implication: Structural conditions now exist for major monthly supply chain campaigns. Mini Shai-Hulud source code is now public. The entry barrier has collapsed.

13. THREAT ACTOR PROFILES

13.1 TeamPCP / UNC6780

Attribute	Assessment
Designation	TeamPCP; aliases: DeadCatx3, PCPcat, PersyPCP, ShellForce, CipherForce
Formal Tracking	UNC6780 (GTIG, 8 April 2026); SANDCLOCK (malware family)
Activity Period	December 2025–May 2026 (confirmed); Shai-Hulud lineage from Sep 2025 (STRONGLY REPORTED)
Classification	Cloud-native cybercriminal; credential theft, access brokering, cryptomining, destructive operations
Collaboration	Vect ransomware (STRONGLY REPORTED, Akamai — active victims published Apr 15, 2026); LAPSUS\$ (CONFIRMED for GitHub data joint sale at \$95K; broader operational merger unconfirmed)
Communication	Telegram @Persy_PCP, @teampcp; public taunting; campaign branding in payloads
Attribution Confidence	HIGH — consistent RSA key, tpcp.tar.gz naming, tpcp-docs pattern, SANDCLOCK string, GTIG formal designation
Nationality / Location	Unconfirmed; Western hemisphere UTC-4 to UTC-9 based on timestamp analysis
C2 Hosting	TECHOFF SRV LIMITED (ASN AS48090), UK-registered bulletproof hosting facade; 236 domains, 57 IPs, Amsterdam
Operational Style	Large-scale automation of known techniques; ego-driven defacement; AI-augmented initial access; source code published publicly
AI Capabilities	hackerbot-claw (OpenCLAW-based agent for autonomous vulnerability scanning and initial access)
Novel Techniques	ICP blockchain C2; first self-propagating npm worm; WAV steganography; Session Protocol C2; credential-free OIDC initial access; first SLSA L3 provenance defeat; platform-level breach (GitHub)
Industry Focus	BFSI, Consumer Goods, Professional Services, DevOps/Security vendors, AI/ML infrastructure, Developer ecosystem

13.2 Lazarus Group / DPRK

Attribute	Assessment
Designation	Lazarus Group (APT38); Sapphire Sleet, BlueNoroff, Famous Chollima, Shifty Corsair
Government	DPRK (Democratic People's Republic of Korea)
Sophistication	HIGH; state-level resources; multi-year sustained campaigns
Contagious Interview	Active since May 2025; Veltrix Capital brand; graphalgo npm campaign

Attribute	Assessment
Key Distinction from TeamPCP	Different C2; different payload families (BeaverTail/InvisibleFerret vs SANDCLOCK); different initial access (social engineering vs CI/CD exploitation)

13.3 PCPJack (Possible ex-TeamPCP Affiliate)

Attribute	Assessment
Designation	PCPJack
Formal Tracking	None assigned; tracked by SentinelLABS
Discovery	April 28, 2026 (VirusTotal K8s hunting rule); Disclosed May 7, 2026
Attribution Confidence	MODERATE — SentinelOne assesses a possible former TeamPCP operator based on tradecraft overlap with the Dec 2025 PCPCat phase and detailed knowledge of TeamPCP artifact paths
Relationship to TeamPCP	Adversarial-competitive: actively evicts TeamPCP artifacts before deploying own tooling; operates on the same victim population; tracks "PCP replaced" metric in exfiltrated data
Motivation	Financial (credential theft, resale, downstream abuse)
Initial Access	CVE exploitation of exposed web applications (Next.js, WordPress plugins, CentOS Web Panel)
Targeting	Cloud infrastructure previously compromised by TeamPCP; exposed Docker/K8s/Redis/MongoDB services
C2	Telegram (encrypted channel); Sliver C2 implant (x86_64, x86, ARM)
Staging	Attacker-controlled Amazon S3: spm-cdn-assets-dist-2026.s3.us-east-2.amazonaws.com
Key Distinction from TeamPCP	Does NOT compromise package registries; targets cloud infrastructure directly via CVE exploitation rather than supply chain poisoning
Defensive Implication	No TeamPCP artifacts ≠ clean system. PCPJack evicts TeamPCP before installing itself. Incident response must explicitly hunt for PCPJack IOCs independently.

14. THREAT ACTOR'S OWN DEFENSIVE GUIDANCE AND ANALYST ASSESSMENT

Source: Public interview with TeamPCP conducted by security researcher Erez Dasa, published at ransomware-interviews.base44.app/interview/teampcp.

14.1 TeamPCP's Five Defensive Recommendations

In the published interview, TeamPCP was asked what defensive measures organizations should take against supply chain attacks. The actor's response — paraphrased — was:

- **Minimum age threshold for libraries.** Only use packages that have existed for some minimum period (suggested: one to several weeks) before allowing installation in production pipelines.
- **Pin versions to a specific hash.** Lock package installations to exact, cryptographically verified versions rather than version ranges or mutable floating references.
- **Least-privilege tokens.** Use tokens with minimum necessary permissions; do not allow developer machines or CI/CD runners to hold credentials with more access than the current task requires.
- **Know and restrict IDE extensions.** Audit which IDE extensions developers use; restrict or allowlist extensions rather than permitting unrestricted Marketplace installation.
- **Use Socket.** The actor specifically endorsed Socket as a tool that "will detect malware before the package even reaches the developer machine" and will publish IOCs and remediation steps.

14.2 Analyst Assessment

Threat actor interviews are a known CTI genre. Advice from an attacker about what stops them is worth taking seriously — but it is not neutral, and the specific vendor endorsement warrants scrutiny. The table below assesses each recommendation against the current threat landscape.

Control	Verdict	Analysis
Minimum age threshold	Factual; mainstream; effective	Most malicious packages are detected and removed within hours to days. A 7–14-day quarantine period blocks the majority of supply chain attacks at a low operational cost. Endorsed by OpenSSF, Snyk, and Phylum. Trade-off: delayed security patches — manageable with a documented override process.

Control	Verdict	Analysis
Hash pinning	Necessary but not sufficient	Lockfile integrity hashes defeat post-publish tampering and mirror poisoning. They do not protect against a malicious package at first pin, nor transitive dependencies, unless the full tree is locked. Would not have stopped TeamPCP: the initial seed credential came from a build environment running the legitimate (non-pinned) scanner.
Least-privilege tokens	Unambiguously correct	The single most consistently applicable control across all campaigns in this report. Aligns with NIST SP 800-204D, SLSA, OWASP CI/CD Top 10. Specific application: separate build from publish credentials; use short-lived OIDC tokens; restrict OIDC subject claims to specific branches and workflows.
IDE extension governance	Factual; severely underappreciated	The Nx Console compromise and the GitHub breach it enabled are the definitive proof. TeamPCP flagged this in their interview and then executed it in their most impactful attack. VS Code and JetBrains extensions execute with full developer user privileges and auto-update without explicit consent. Organizations with strong CI/CD controls but unrestricted extension installation have a significant unaddressed attack surface.
Socket (pre-install scanning)	Technically sound; claim overstated	Socket is legitimate and well-regarded. However, "will detect the malware before the package even reaches your machine" is vendor-grade phrasing that overstates any single tool's capability. Mini Shai-Hulud produced the first packages with valid SLSA Build Level 3 provenance, defeating signature-based trust signals. The TanStack attack propagated too quickly for the scanner to detect and prevent the initial wave. Treat it as one layer in a defense-in-depth stack, not a guarantee.

Meta-Assessment: TeamPCP's five recommendations align with OpenSSF, SLSA, and CISA software supply chain guidance — the correct benchmark. The single-vendor endorsement is the element warranting the most scrutiny. Regardless of the actor's intent, no single detection tool should be the sole control layer. The entire TeamPCP campaign — SANDCLOCK to CanisterWorm to Mini Shai-Hulud — is engineered to stay ahead of static detection by rotating payloads (ICP canister), masquerading as legitimate traffic (fake OTel), using decentralized C2 (Session Protocol), and forging trust signals (SLSA provenance). Defense in depth is not optional.

Retrospective Note — IDE Extensions: TeamPCPs' advice to restrict IDE extensions in the interview, followed by their trojanization of Nx Console and the subsequent GitHub breach, is the strongest possible empirical validation of that recommendation. Whether intentional or not, the actor described the defensive gap and then immediately exploited it.

15. IMPACT ASSESSMENT AND COMPARATIVE ANALYSIS

15.1 TeamPCP Campaign vs. SolarWinds vs. Log4j

Dimension	SolarWinds (2020)	Log4j (2021)	TeamPCP 2026 Campaign
Initial Compromise	Trojanized Orion binary	Vulnerable logging library	Security tooling + CI/CD + IDE extensions + GitHub platform
Distribution Breadth	~18,000 customers received tainted update	100M+ instances globally	1,000+ confirmed SaaS; 5,000–10,000 projected
Sophistication	HIGH (selective espionage implant)	MODERATE (universal vulnerability)	HIGH (AI-assisted; OIDC-credential-free; autonomous worm; SLSA defeat)
Follow-on Impact	~100 selectively compromised (espionage)	Widespread indiscriminate RCE	Cloud intrusion (1,000+); GitHub breach; European Commission; OpenAI signing certs rotated
Trust Anchors Broken	Network management privilege	Logging infrastructure (universal)	Security tooling + CI/CD + package registries + IDE extensions + GitHub platform
Platform Breach	None	None	GitHub internal repositories (~3,800 private repos)
SLSA Defeat	N/A	N/A	First SLSA Build Level 3 provenance defeat (TanStack, May 2026)

15.2 Why the GitHub Breach Represents a Qualitative Escalation

SolarWinds and Log4j compromised software that organizations ran. The TeamPCP campaign, via Nx Console → GitHub employee → GitHub internal, has compromised the platform that manages and distributes software at a global scale. GitHub controls trust relationships for billions of repositories, the GitHub Actions marketplace, and authentication infrastructure for git operations worldwide. Exfiltrated internal source code enables offline vulnerability research against the platform itself, compounding risk over time as the actor analyzes code for exploitable conditions.

15.3 Victim Risk Tiers

Tier 1 — Highest Risk: Software vendors; OSS maintainers; system integrators and MSPs; government contractors; organizations with high-privilege CI/CD access to cloud production environments.

Tier 2 — Medium Risk: Large enterprises with multiple development teams; SaaS providers; cryptocurrency and blockchain platforms; AI/ML infrastructure teams.

Tier 3 — Lower but Material Risk: Individual developers who executed poisoned packages; organizations that quickly pinned or remediated affected packages.

16. RECOMMENDATIONS

16.1 Immediate Actions (0–48 Hours)

- **Treat as credential exposure first.** Rotate ALL reachable secrets atomically: GitHub PATs, cloud credentials (AWS/GCP/Azure), Kubernetes tokens, SSH keys, Docker auth, npm/PyPI tokens, Terraform state. Revoke before deploying replacements through an out-of-band channel.
- **Scope by exact artifacts and exposure windows.** Hunt for: Trivy v0.69.4–0.69.6; LiteLLM 1.82.7–1.82.8; telnyx 4.87.1–4.87.2; PyTorch Lightning 2.6.2–2.6.3; axios 1.14.1/0.30.4; all affected @tanstack/* versions (GHSA-g7cv-rxg3-hmpx for full list); SAP CAP packages (April 29 window); Bitwarden CLI (April window).
- **Audit IDE extensions immediately.** Inventory all installed VS Code and JetBrains extensions. Remove any not on an approved allowlist. The Nx Console incident confirms high-install-count verified-publisher extensions are now targeted attack surfaces.
- **Search GitHub organizations for tcp- docs / docs-tcp repositories** — confirmed fallback exfiltration indicator.
- **Hunt persistence artifacts.** Linux: pgmon.service, pgmonitor.service, internal-monitor.service, audiomon.service, gh-token-monitor.service in systemd directories; /tmp/pglog, /tmp/pg_state. macOS: ~/Library/LaunchAgents/com.user.gh-token-monitor.plist. Unexpected .claude/ and .vscode/ directory contents in any repository.
- **Block C2 at the DNS/proxy level.** Block: git-tanstack[.]com, *.getsession.org, scan.aquasecurity[.]org, checkmarx[.]zone, models.LiteLLM[.]cloud. Monitor for icp0.io canister subdomains.
- **Cloud audit.** Scan CloudTrail for iam:CreateAccessKey, sts:GetCallerIdentity, IAM enumeration events during exposure windows from unexpected source IPs. Search requestParameters for the strings "pawn" or "massive-exfil".
- **Kubernetes audit.** kubectl get daemonsets --all-namespaces; search for host-provisioner-, node-setup-, kamikaze container names.
- **Rotate twice if compromised.** The first rotation may have used compromised infrastructure (the Aqua failure model). Perform a second atomic rotation from a clean out-of-band environment.

16.2 Short-Term Actions (1–2 Weeks)

- **Pin GitHub Actions to immutable commit SHAs.** Not mutable version tags. Single most effective control against tag-poisoning attacks.
- **Restrict OIDC token scopes in GitHub Actions.** Set `permissions: id-token: none` in all workflows that do not publish. Restrict `id-token: write` to only the specific release job; tightly constrain subject claims (specific branch, not wildcard). Block fork PRs from writing to workflow caches — the exact TanStack vector.
- **Implement IDE extension allowlisting.** Maintain an approved extension list; block unlisted installations. Treat the VS Code marketplace as untrusted code execution.
- **Apply TeamPCP's own advice:** minimum age threshold (7–14 days) on new package versions; hash pinning across all package managers; least-privilege tokens throughout CI/CD.
- **Pin container images to digest + provenance verification.** Not tag-based.
- **Disable npm postinstall hooks by default.** `npm config set ignore-scripts true`. Enable selectively per project.
- **Full dependency audit** including transitive tree. Pay special attention to `.pth` files and pre- and post-install hooks.
- **CI/CD network egress allowlisting.** Outbound network restrictions on build runners; block IMDS or enforce IMDSv2 with hop limits.
- **Enable runtime detection on CI runners** (Sysdig Secure, Falco). Detect credential theft, data exfiltration, and process memory access.

16.3 Medium-Term Actions (1–3 Months)

- **Adopt SLSA Level 3+ with the correct mental model.** SLSA L3 provenance is not a trust anchor when the build environment is compromised — TanStack proved this. SLSA must be combined with pipeline access controls (restricted OIDC, branch/tag protection), build environment hardening (ephemeral runners, egress allowlists), and a separate publish identity (manual approval gate on release).
- **Implement Trusted Publisher for PyPI.** OIDC-based publishing; eliminate long-lived tokens.

- **Monitor Certificate Transparency logs for pre-operation indicators.** TeamPCP provisioned TLS certificates 1–2 days before each attack phase. Monitor via crt.sh or Certstream for typosquat domains mimicking your brand or your security vendors' brands.
- **Enforce commit signing and tag protection** via GitHub rulesets. Require signature verification on tag operations; restrict tag creation/update/deletion; monitor tag mutation events.
- **Prepare for worm source code copycats.** Mini Shai-Hulud source published May 12, 2026. Future attacks using the same toolchain may not be TeamPCP. Defensive controls remain identical; attribution confidence will decrease.
- **Monitor for Session Protocol / getsession.org C2.** Block *.getsession.org at the DNS/proxy level. This decentralized C2 pattern is expected to be adopted by additional threat actors.
- **Tabletop exercises** using TeamPCP TTP playbook (credential chaining, tag poisoning, worm propagation, cache persistence, OIDC abuse, IDE extension delivery) as threat model.

17. WEAKNESSES BYPASSED — PRACTITIONER ANALYSIS

This section is for defenders. For each major attack pattern in this campaign period, it documents the specific assumption that was broken, the control that was defeated, and the minimum viable fix that would have materially degraded or stopped the attack. Knowing what failed is more actionable than knowing what happened.

WB-1 — Mutable Git Tags as Version Pinning

Attack instances: tj-actions (Mar 2025), TeamPCP Trivy/Checkmarx (Mar 2026)

The broken assumption: "If I pin my GitHub Actions workflow to `uses: vendor/action@v1.2.3`, I am using the code I reviewed."

What actually happened: Git tags are mutable pointers. Any actor with push access can silently redirect a tag to a completely different commit. There is no transparency log, no notification to consumers, and no visual change in workflow files. All 76 trivy-action version tags were rewritten in under an hour. No downstream pipeline saw any change to its YAML.

Control defeated: Version-pinned workflow references. This is widely recommended as a security best practice. It is not sufficient when the tag itself can be rewritten.

Minimum viable fix: Pin to the full immutable commit SHA, not the tag name. `uses: aquasecurity/trivy-action@a20de5420d57c4102486cdd9349b532bf5b12c4f` cannot be silently redirected. Combine with GitHub rulesets enforcing signature verification on tag operations. Monitor tag mutation events via the GitHub API audit log.

Who this applies to: Every organization using GitHub Actions with mutable version references — which is most of the industry.

WB-2 — `pull_request_target` as Trusted Execution Context

Attack instances: Ultralytics (Dec 2024), hackerbot-claw/Trivy (Feb 2026), TanStack cache poisoning (May 2026)

The broken assumption: "Pull requests from forks are untrusted; my repository secrets are protected."

What actually happened: The `pull_request_target` trigger runs with the *base repository's* secrets and permissions, even when processing code from a fork. Combined with unsanitized user-controlled input (branch names, PR titles) interpolated into shell commands, or with cache writes accessible to the workflow, this creates a path from a fork pull request to execution with

full repository privileges. The attack is entirely legitimate within GitHub's design — there is no exploit. It is a misconfiguration of a dangerous-by-default feature.

Control defeated: GitHub's default trust boundary between fork and base repository. The boundary is real for `pull_request` but intentionally absent for `pull_request_target`.

Minimum viable fix (three controls, all required together):

- Never use `pull_request_target` unless you fully understand its permission model and have explicitly audited the workflow
- Never interpolate user-controlled values (branch names, PR titles, commit messages) directly into shell commands in any workflow
- Block fork PRs from writing to the shared Actions cache: set `cache-write: false` or equivalent in cache action configurations

WB-3 — Official Namespace as Trust Signal

Attack instances: LiteLLM, Telnyx, axios, PyTorch Lightning, SAP CAP, Bitwarden CLI

The broken assumption: "If the package comes from the official publisher's namespace on npm or PyPI, it is safe to install."

What actually happened: The attacker stole publisher credentials from CI/CD environments (via the earlier Trivy compromise) and then used them to publish malicious versions from legitimate publisher accounts. The malicious packages were indistinguishable from legitimate ones by namespace, publisher identity, download history, or registry security signals. This is not a typosquatting attack or a dependency confusion attack — it is the real package from the real account.

Control defeated: Package registry namespace trust. This is the foundational assumption of the modern package manager model. It held until it didn't.

Minimum viable fix:

- Minimum age threshold: 7–14 days before new package versions are permitted in production (most malicious versions are detected within hours to days)
- Hash pinning in lockfiles: pins the exact bytes, not just the version string — protects against post-publication tampering even from the official account
- Runtime behavioral detection: a scanner catching the payload behavior after install is a second detection layer when the namespace signal fails

WB-4 — SLSA Build Level 3 Provenance as Trust Anchor

Attack instances: TanStack / Mini Shai-Hulud (May 2026) — **first documented defeat**

The broken assumption: "If a package has a valid SLSA Build Level 3 provenance attestation from a GitHub Actions runner, it was built from reviewed source code in a controlled environment."

What actually happened: The attacker compromised TanStack's GitHub Actions CI pipeline via OIDC token theft + cache poisoning, then published malicious packages from within the legitimate runner. Because the attacker was executing inside the legitimate CI environment, the OIDC tokens were genuine, the Sigstore attestation was genuine, and the SLSA L3 provenance was genuine. The attestation correctly described what happened — a build executed in the configured environment — but that environment was already under attacker control.

Control defeated: Sigstore/cosign provenance attestation. SLSA L3. The first time in documented history that a valid SLSA L3 attestation accompanied malicious packages.

Minimum viable fix: SLSA provenance is not a sole trust anchor. It must be combined with:

- Constrained OIDC subject claims: `sub: repo:org/repo:ref:refs/heads/main` — not wildcard subjects
- Separated publish identity: publish step requires a manual approval gate; automated CI cannot publish without human sign-off
- Block fork PRs from writing to the cache used by the publish workflow
- Monitor for anomalous publish events: same package published twice in quick succession, unusual publish time, or publish from an unexpected CI run

WB-5 — GitHub Actions Cache Isolation

Attack instances: TanStack (May 2026)

The broken assumption: "GitHub Actions cache entries are isolated; a fork's pull request cannot poison the cache used by the base repository's release workflow."

What actually happened: A pull request from a fork triggered a workflow with write access to the base repository's cache. The attacker's code poisoned that cache entry and waited. When the legitimate release workflow subsequently ran and read the poisoned cache entry, it executed attacker-controlled code with the full privileges of the release pipeline — including OIDC token issuance for npm publish.

Control defeated: GitHub Actions cache trust boundary. The cache is shared across workflow runs by default; the access controls are not obvious and are misconfigured in many repositories.

Minimum viable fix: Explicitly turn off cache writes in workflows that fork pull requests can trigger. Use separate cache keys for trusted and untrusted workflow runs. Audit all workflows using `pull_request_target` for cache interactions.

WB-6 — IDE Extension Safety Signals

Attack instances: Nx Console (May 2026) → GitHub internal breach

The broken assumption: "A VS Code extension with 2.2 million installs, a verified publisher badge, and a long history of legitimate use is safe to install and auto-update."

What actually happened: TeamPCP trojanized the Nx Console extension and published the malicious version to the Visual Studio Marketplace. The extension's verified publisher badge, high install count, and established reputation all remained intact because they reflect the publisher's history — not the current version's content. The auto-update mechanism pushed the malicious version to every active user within the 18-minute window without any user interaction. One GitHub employee's device was compromised this way, leading to the breach of 3,800 internal repositories.

Control defeated: Marketplace safety signals (install count, verified publisher, ratings). These signals are backward-looking; they validate the publisher's past reputation, not the current release.

Minimum viable fix: Implement an organization-wide IDE extension allowlist. Block installation of any extension not on the approved list. Treat the VS Code Marketplace as untrusted code execution — equivalent to running an unsigned binary as the current developer user. Review and re-approve allowlisted extensions after any version update.

WB-7 — Credential Isolation in CI/CD Runner Memory

Attack instances: TeamPCP Trivy phase (Mar 2026) — first documented use of `/proc/mem` against CI runners

The broken assumption: "Secrets injected into GitHub Actions workflows via the `secrets` context are protected. They do not appear in logs, env vars, or filesystem paths. They are safe."

What actually happened: The malicious Trivy binary targeted the Runner.Worker processes by reading `/proc/<pid>/mem` directly, extracting the raw process memory. Secrets marked `isSecret: true` in GitHub's runner implementation exist in memory during workflow execution. Memory scraping bypasses every protection layer that prevents secrets from appearing in logs, environment variables, or files — because it reads the memory directly, before any of those protection layers apply.

Control defeated: GitHub Actions secret masking, log scrubbing, and environment isolation. These controls prevent secrets from being *logged*; they do not prevent secrets from being *read from memory*.

Minimum viable fix: Ephemeral, network-restricted build runners. A build runner that cannot make outbound connections to the attacker infrastructure cannot exfiltrate secrets even if the payload executes. Combine with: runtime behavioral detection (Falco, Tetragon) monitoring for `/proc/$PID/mem` access from non-root processes; `pgrep -f Runner.Worker` process enumeration patterns; anomalous outbound HTTP POST from CI agents.

WB-8 — Non-Atomic Credential Rotation During Active Breach

Attack instances: Aqua Security February → March 2026 breach chain (the rotation failure that enabled Phase 2)

The broken assumption: "We identified the breach, rotated our credentials, and the incident is contained."

What actually happened: Aqua Security's post-February-2026 credential rotation followed a standard zero-downtime pattern: issue new credentials → update systems → verify → revoke old credentials. During this sequential process, the new credentials were deployed through the infrastructure that the attacker still had access to. The attacker observed the new credentials, retained access, and waited 18 days before executing the March 19 attack using the newly rotated tokens.

Control defeated: Standard credential rotation procedure. The "rotate credentials to contain a breach" playbook is correct in theory, but fails in practice when the attacker can observe the rotation through the same compromised infrastructure.

Minimum viable fix: Incident-response rotation requires a different procedure from routine rotation:

- **Immediately revoke** all suspected credentials — accept the service disruption; downtime is better than continued access
- **Generate replacements** through an out-of-band channel that the attacker cannot observe (separate admin console, clean workstation, not the compromised CI pipeline)
- **Deploy replacements** only after confirming the attacker's access path has been eliminated
- **Monitor** for authentication attempts using revoked credentials after rotation is complete

WB-9 — Security Tooling as Trusted Execution Context

Attack instances: Trivy scanner, Checkmarx KICS, Checkmarx Jenkins AST plugin

The broken assumption: "Security scanning tools run with elevated trust in our pipeline because they need to scan our code and infrastructure. Compromise of a scanner is a remote risk."

What actually happened: Trivy is widely deployed in privileged CI/CD positions precisely because it needs to scan credentials, containers, and infrastructure. It runs with access to cloud credentials, Docker sockets, Kubernetes config, and the GitHub token. When Trivy was compromised, it ran silently alongside the legitimate scan (with normal output and no errors) while exfiltrating everything it could access. Organizations with the most thorough security scanning practices had the largest attack surface.

Control defeated: The implicit elevated trust granted to security tooling. This creates what researchers described as a "trust inversion problem" — the more security-conscious the organization, the more exposed it was.

Minimum viable fix: Treat security tooling supply chains with stricter controls than application supply chains — not looser ones, because compromise delivers attacker code into the most privileged pipeline positions. Pin all scanner versions to verified digests. Run scanners in isolated environments with egress restrictions. Separate scanner execution credentials from deployment credentials. Monitor scanner processes for unexpected network activity.

18. LESSONS LEARNED

Synthesis across all campaigns, 2024–2026. These are not individual incident postmortems — they are cross-cutting structural failures that recur across different threat actors, ecosystems, and organizations.

LL-1: Every Trust Signal Has Now Been Defeated at Least Once

Trust Signal	First Defeated	How
Package namespace (official publisher)	2021 (ua-parser-js)	Credential theft → malicious publish
Version-pinned GitHub Actions tag	2025 (tj-actions)	Mutable tag rewrite
pull_request_target isolation	2024 (Ultralytics)	Workflow misconfiguration
SLSA Build Level 3 provenance	2026 (TanStack)	Pipeline access → valid attestation from attacker-controlled run
Verified publisher + high install count (IDE)	2026 (Nx Console)	Extension trojanization
Security scanner as a safe execution context	2026 (Trivy)	Compromised scanner runs with pipeline privileges

Implication for practitioners: No single trust signal is a sufficient control. Defense-in-depth is not a buzzword; it is a structural requirement, given that every layer is bypassable. Ask yourself: "If this trust signal fails, what is my next layer of detection or containment?"

LL-2: The Tools You Trust Most Are the Highest-Value Targets

Security scanners, AI gateways, password manager CLIs, and IDE extensions all run with elevated privilege or hold sensitive credentials. Attackers have figured this out. Trivy runs in privileged CI/CD positions. LiteLLM holds API keys for every LLM provider. Bitwarden CLI stores vault credentials. Nx Console executes as the developer user with full filesystem access.

Implication: The security tooling supply chain requires *stricter* vetting than the application supply chain — not equal vetting, and definitely not looser. The tools that protect you are now the attack surface.

LL-3: Credential Rotation Fails Unless It Is Atomic

The Aqua Security non-atomic rotation is a documented failure mode that directly enabled the March 2026 attacks. The standard zero-downtime rotation pattern — which is recommended everywhere for operational continuity — is dangerous during an active breach.

Implication: Organizations need two rotation playbooks: one for planned rotation (zero-downtime, sequential) and one for incident-response rotation (atomic, accept downtime, out-of-band). Most organizations only have the first. The second is the one that matters during a breach.

LL-4: The Worm's Source Code Being Public Is Structural, Not an Incident

TeamPCP publishing Mini Shai-Hulud's source code on May 12, 2026, followed by a formal contest to use it, is not a story that ends. The automation logic, deployment patterns, persistence concepts, and C2 infrastructure designs are now available to any actor. Four cpycat packages were confirmed within days. Future attacks using the Shai-Hulud toolchain should not be attributed to TeamPCP without additional corroborating indicators.

Implication: Defensive controls must be designed assuming the attack toolchain is a commodity. The question is no longer "is the adversary sophisticated enough to do this?" — the answer is "yes, because the toolchain is now free."

LL-5: Decentralized and Blockchain C2 Makes Takedown Ineffective

TeamPCPs' use of ICP blockchain canisters (CanisterWorm) and Session Protocol decentralized messaging (Mini Shai-Hulud) means traditional incident response — domain takedown, C2 IP blocking — does not work. The canister cannot be taken down. Any single entity controls the Session network.

Implication: Takedown-oriented incident response must be replaced with payload-rotation detection (monitoring for canister URL changes), network behavioral detection (blocking *.getsession.org, monitoring for icp0.io traffic), and endpoint-side containment (killing persistence artifacts before the C2 re-arms).

LL-6: IDE Extensions Are Production-Grade Attack Surface

The Nx Console incident is the proof case. A VS Code extension with 2.2 million installs, running as the developer user, auto-updated without consent, and, within 18 minutes, compromised enough employee devices to breach GitHub's internal repositories. Most organizations have no extension governance whatsoever.

Implication: IDE extensions should be treated under the same governance as production software deployments. Allowlisting is not optional for any organization where developer workstations have access to sensitive credentials, internal systems, or deployment infrastructure.

LL-7: Rival Actors Competing on Your Victim Population Is a New Threat Class

PCPJack's deliberate eviction of TeamPCP from compromised systems creates a scenario defenders have not encountered before: two distinct criminal actors competing for the same machines. A system showing no TeamPCP artifacts may simply be running PCPJack instead. Incident response teams declaring a system "clean" after removing TeamPCP artifacts may be declaring the wrong result.

Implication: Threat hunting on systems potentially exposed to TeamPCP must explicitly include PCPJack indicators. The absence of one threat actor's artifacts does not constitute evidence of a clean system when a rival is actively evicting that actor.

LL-8: Gamified Attacks Scale Beyond What Any Single Actor Could Achieve

The BreachForums contest weaponizes the open-sourced toolchain at scale. Instead of TeamPCP needing to find and exploit individual packages, the contest creates a community of operators, each finding their own targets and contributing to the ecosystem-wide credential harvest. The \$1,000 Monero prize is economically insignificant; the reputation and access incentives are the real drivers.

Implication: The volume of supply chain attacks will increase, driven by new actors enabled by open-sourced tooling and community gamification. Organizations relying on low attack volume as a proxy for low risk are operating on an assumption that no longer holds.

19. CALL TO ACTION — A MESSAGE TO THE OSS ECOSYSTEM

This section is addressed directly to the people who build, maintain, and consume open-source software. Not to CISOs or compliance teams. To maintainers, developers, platform operators, and the community.

To OSS Maintainers

You are the people these attacks target first. Your publishing tokens, your CI/CD credentials, and your GitHub tokens are the primary attack surface. The campaigns documented in this report are not going to stop — they are accelerating. PCPJack, the BreachForums contest, and the public worm source code mean the tools to attack you are now free, and the incentive structure for doing so has never been stronger.

What you can do today, before the next attack:

- Enable 2FA with a hardware key on every package registry account you hold. Not TOTP. A hardware key.
- Adopt Trusted Publishing (OIDC-based publishing) on npm, PyPI, crates.io, and RubyGems. Eliminate long-lived tokens from your CI/CD entirely.
- Pin your own CI/CD dependencies — including security scanners — to commit SHAs. If your scanner is compromised, your pipeline becomes the attack.
- Rotate your publish tokens now, atomically, from a clean machine. Not because you are compromised, but because the cost of rotating is an hour, and the cost of not rotating when you need to is your project and your users.
- Review every GitHub Actions workflow for `pull_request_target` usage. If you have it and cannot justify why, remove it.

The maintainer community is the first and most important line of defense in the OSS supply chain. The tools and platforms exist to make this defensible. Most maintainers simply have not had time to implement them. The attacks documented here are a reason to make time.

To Platform Operators (GitHub, npm, PyPI, Docker Hub, VS Code Marketplace)

The architectural decisions that enabled these attacks — mutable Git tags, `pull_request_target` default permissions, shared workflow caches across trust boundaries, marketplace auto-updates without human review — are design decisions, not bugs. Dan Lorenc's characterization is precise: some of these design decisions are "plain irresponsible today and ignore a decade of supply chain security work from other ecosystems."

Specific asks from practitioners:

- **GitHub:** Immutable tags by default for repositories above a minimum download/usage threshold. Transparency logs for tag mutations—mandatory SHA pinning in Marketplace-listed Actions. Cache write isolation between trusted and untrusted workflow runs is a default, not an opt-in.
- **npm:** Mandatory 2FA with hardware keys for all packages above 10,000 weekly downloads. Transparency log for all publish operations. Supply chain attack "cooling period" where new versions of high-download packages are held for human review before propagation.
- **PyPI/crates.io/RubyGems:** Expand adoption incentives for Trusted Publishing. Provide automated detection of credential-stuffing publish patterns (same credentials used from new IPs, burst publishing of multiple packages). Surface canary alerts to maintainers when their packages are published from unexpected CI environments.
- **VS Code Marketplace:** Implement content-based diff scanning of extension updates before they propagate to auto-update. Hold updates to extensions with more than 100,000 installs for 24 hours with automated behavioral analysis. Allow organizations to pin extension versions and centrally block auto-updates.

The GitHub breach is proof that platform-level architectural decisions have ecosystem-scale consequences. A single mutable tag in a widely used Action, a single cache boundary misconfiguration, and a single VS Code extension auto-update — these are individual design decisions that collectively enabled the breach of the platform hosting 100 million developers' code.

To Enterprise Consumers of OSS

Your supply chain is not someone else's problem. The European Commission, OpenAI, Mercor, and GitHub are all confirmed downstream victims. They were not targeted for who they are — they were compromised because they used the same packages and tools as everyone else.

What changes immediately:

- The answer to "Do we use this package?" is no longer sufficient. The answer needed is "do we use this package, pinned to a reviewed version, with behavioral monitoring at install time and runtime?"
- Your developer workstations are production endpoints. Treat them that way. EDR, extension allowlisting, and secret management hygiene on developer machines are not developer-experience problems — they are security infrastructure.
- Your CI/CD pipelines hold cloud production credentials. Map every secret to every pipeline that holds it. Know what an attacker would do with each one. The European Commission case is the model: one stolen AWS key, 340 GB of data, 71 organizational victims.
- Vendor your critical dependencies. Maintain a private mirror of high-risk packages at pinned versions. Accept the maintenance cost. It is smaller than the incident response cost.

To Security Researchers and the CTI Community

The campaigns documented here are accelerating faster than they are being documented. The PCPJack discovery happened because one researcher had a VirusTotal hunting rule in exactly the right place. The TanStack attack was detected because Socket's scanner flagged something in the package within six minutes. The Nx Console compromise lasted 18 minutes — the detection window for the entire GitHub breach was 18 minutes wide.

The community needs:

- Faster IOC publication. The delay between incident detection and the public release of IOCs needs to be reduced. Organizations defending against Mini Shai-Hulud need C2 domains and file hashes within hours, not days.
- Unified IOC aggregation. TeamPCP, Lazarus, PCPJack, and the BreachForums contest copycats are all operating in the same ecosystem. A practitioner should not need to read twelve separate vendor blogs to build a complete blacklist.
- Attribution discipline. When source code is public, and contest entries are being deployed, attributing "Shai-Hulud-style" attacks to TeamPCP muddies the threat picture. The toolchain is a commodity; the actor is not always TeamPCP.
- Government advisory alignment. CVE-2026-45321 (TanStack) is not in the CISA KEV as of this report's publication date, despite representing the first SLSA L3 provenance defeat and enabling the GitHub breach. Government advisory cadence needs to match the attack cadence.

The Bottom Line

The OSS supply chain is a shared global resource. It is also the attack surface that enables the fastest lateral movement from any single compromised package to any organization in the world. The campaigns documented in this report are not anomalies — they are a preview of the normal operating environment.

The technical controls exist. The frameworks exist. The tooling exists. What has been missing is urgency.

TeamPCP, Lazarus, PCPJack, and the BreachForums contest participants are operating on the assumption that the ecosystem moves slowly and defends reactively. They are right, and that has to change.

Every unmaintained `pull_request_target` workflow is an open door. Every unpinned package version is an unverified assumption. Every unrotated publishing token is an uncontained risk. Every unallowlisted IDE extension is a loaded weapon on a developer's workstation.

The question is not whether the next attack is coming. It is whether the next attack finds the same doors open that the last one found.

20. SUMMARY

This report covers the most consequential period in the history of open-source software supply chain security. From Ultralytics (December 2024) through the GitHub internal breach (May 2026), a progression of threat actors, techniques, and target classes culminated in the compromise of GitHub itself — the platform on which virtually all open-source software is developed and distributed.

The attack trajectory followed a consistent logic: find a trusted component, compromise its publication channel, harvest the credentials of everyone who consumed it, and use those credentials to poison the next trusted component. Each campaign generation closed the detection gaps exposed by its predecessor. The techniques are now commoditised — the worm source code is public, a criminal contest is actively recruiting new operators, and a rival worm is competing for the same victim population.

The practitioner sections of this report (Sections 17–19) are the operational core: nine broken assumptions with minimum viable fixes (Section 17), eight cross-campaign structural lessons (Section 18), and a direct call to action addressed to every stakeholder in the OSS ecosystem (Section 19). Read those sections. Act on them.

Campaign Status as of 22 May 2026: ACTIVE. Worm toolchain is public. BreachForums contest is recruiting. PCPJack is operating on previously compromised systems. Additional targets across RubyGems, crates.io, and Maven Central are assessed as likely. Active hunting is required — cache persistence means remediated environments may still contain compromised artifact.

21. REFERENCES

Primary Vendor Advisories

- Aqua Security. Trivy Supply Chain Attack.
<https://www.aquasec.com/blog/trivy-supply-chain-attack-what-you-need-to-know/>
- Docker. Trivy Supply Chain Compromise: What Docker Hub Users Should Know.
<https://www.docker.com/blog/trivy-supply-chain-compromise-what-docker-hub-users-should-know/>
- Checkmarx. Checkmarx Security Update.
<https://checkmarx.com/blog/checkmarx-security-update/>
- LiteLLM. Security Update March 2026.
<https://docs.litellm.ai/blog/security-update-march-2026>
- Telnyx. Python SDK Supply Chain Security Notice.
<https://telnyx.com/resources/telnyx-python-sdk-supply-chain-security-notice-march-2026>
- Mistral AI. Advisory MAI-2026-002 (TanStack worm). May 12, 2026.
- GitHub Advisory GHSA-69fq-xp46-6x23 — Trivy Ecosystem Supply Chain Compromise.
- GitHub Advisory GHSA-g7cv-rxg3-hmpx — TanStack Supply Chain Compromise (CVE-2026-45321).

Government and Regulatory Advisories

- CISA. CVE-2026-33634 Added to KEV Catalogue (26 March 2026).
<https://www.cisa.gov/>
- CISA. tj-actions/changed-files Compromise (CVE-2025-30066, 18 March 2025).
<https://www.cisa.gov/>
- CERT-EU. European Commission Cloud Breach (2–3 April 2026).
<https://www.cert.europa.eu/>

Security Vendor Research — TeamPCP

- Wiz. Trivy Compromised by TeamPCP.
<https://www.wiz.io/blog/trivy-compromised-teampcp-supply-chain-attack>
- Wiz. Mini Shai-Hulud Strikes Again: TanStack.
<https://www.wiz.io/blog/mini-shai-hulud-strikes-again-tanstack-more-npm-packages-compromised>
- Datadog Security Labs. LiteLLM and Telnyx Compromised on PyPI.
<https://securitylabs.datadoghq.com/articles/LiteLLM-compromised-pypi-teampcpsupply-chain-campaign/>
- Sysdig. TeamPCP Expands: Trivy to Checkmarx.
<https://www.sysdig.com/blog/teampcp-expands-supply-chain-compromise/>
- Microsoft Security Blog. Detecting and Defending Against the Trivy Supply Chain Compromise. <https://www.microsoft.com/en-us/security/blog/2026/03/24/>

- Snyk. Poisoned Security Scanner Backdooring LiteLLM. <https://snyk.io/articles/poisoned-security-scanner-backdooring-LiteLLM/>
- Snyk. TanStack npm Packages Hit by Mini Shai-Hulud. <https://snyk.io/>
- Aikido Security. TeamPCP Deploys CanisterWorm on NPM. <https://www.aikido.dev/blog/teampcp-deploys-worm-npm-trivy-compromise>
- Aikido Security. Telnyx PyPI Compromised. <https://www.aikido.dev/blog/telnyx-pypi-compromised-teampcp-canisterworm>
- Aikido Security. Nx Console VS Code Extension Compromise. <https://www.aikido.dev/blog/>
- JFrog Security Research. CanisterWorm. <https://research.jfrog.com/post/canisterworm/>
- JFrog Security Research. Telnyx Popular Library Hit. <https://research.jfrog.com/post/team-pcp-strikes-again-telnyx-popular-library-hit/>
- Socket. CanisterWorm: npm Publisher Compromise. <https://socket.dev/blog/canisterworm-npm-publisher-compromise>
- Endor Labs. TeamPCP Isn't Done. <https://www.endorlabs.com/learn/teampcp-isnt-done>
- SafeDep. Malicious Telnyx PyPI Compromise. <https://safedep.io/malicious-telnyx-pypi-compromise>
- Semgrep. TanStack Router and Mini Shai-Hulud. <https://semgrep.dev/blog/>
- Trend Micro. Inside the LiteLLM Supply Chain Compromise. <https://www.trendmicro.com/>
- StepSecurity. hackerbot-claw Exploiting GitHub Actions. <https://www.stepsecurity.io/blog/hackerbot-claw-github-actions-exploitation>
- StepSecurity. Mini Shai-Hulud Is Back: TanStack npm. <https://www.stepsecurity.io/blog/mini-shai-hulud-is-back/>
- Akamai. Mini Shai-Hulud: The Worm Returns and Goes Public. <https://www.akamai.com/blog/>
- Palo Alto Networks Unit 42. Trivy Supply Chain Attack. <https://www.paloaltonetworks.com/blog/cloud-security/trivy-supply-chain-attack/>
- CrowdStrike. From Scanner to Stealer: trivy-action Compromise. <https://www.crowdstrike.com/en-us/blog/>
- Arctic Wolf. TeamPCP Supply Chain Attack Campaign. <https://arcticwolf.com/resources/blog/>
- Kaspersky. Trojanization of Trivy, Checkmarx, and LiteLLM. <https://www.kaspersky.com/blog/>
- SANS Institute. When the Security Scanner Became the Weapon. <https://www.sans.org/webcasts/>
- Cyble. TeamPCP Threat Actor Profile. <https://cyble.com/threat-actor-profiles/teampcp/>
- Datadog. TeamPCP IOC Feed (CSV). <https://github.com/DataDog/indicators-of-compromise/raw/refs/heads/main/teampcp/iocs.csv>
- Team Cymru. TECHOFF SRV LIMITED Bulletproof Hosting. <https://www.team-cymru.com/post/jingle-shells/>
- Chainguard (Dan Lorenc). What the Fork? Imposter Commits in GitHub Actions. <https://www.chainguard.dev/unchained/what-the-fork>
- Corgea. Mini Shai-Hulud Supply-Chain Worm IOCs. <https://www.corgea.com/>

- Tenable. Mini Shai-Hulud FAQ. <https://www.tenable.com/>
- ry-allan. tanstack-compromise-checker (detection tool). <https://github.com/ry-allan/tanstack-compromise-checker>
- Help Net Security. GitHub Internal Breach via Nx Console. <https://www.helpnetsecurity.com/>
- Rami McCarthy. TeamPCP Campaign Timeline. <https://ramimac.me/trivy-teampcp/>
- Boost Security Labs. MegaGame10418 and hackerbot-claw. <https://labs.boostsecurity.io/articles/megagame10418/>

Security Vendor Research — Lazarus / DPRK

- ReversingLabs. graphalgo: DPRK Fake Recruiter Campaign. <https://www.reversinglabs.com/>
- Socket.dev. North Korea's Contagious Interview Campaign. <https://socket.dev/supply-chain-attacks/>
- GBHackers. Lazarus Group graphalgo npm packages. <https://gbhackers.com/lazarus-groups-graphalgo/>
- The Hacker News. PyTorch Lightning and Intercom Supply Chain Attacks. <https://thehackernews.com/>
- The Register. Supply chain attacks worm into SAP npm packages. <https://www.theregister.com/>

Security Vendor Research — Historical Campaigns

- Wiz. s1ngularity: Supply Chain Attack Leaks Secrets on GitHub. <https://www.wiz.io/blog/s1ngularity-supply-chain-attack>
- Wiz. Shai-Hulud 2.0: Ongoing Supply Chain Attack. <https://www.wiz.io/blog/shai-hulud-2-0-ongoing-supply-chain-attack>
- GitGuardian. Shai-Hulud 2.0: 33,185 Secrets Exposed. <https://blog.gitguardian.com/shai-hulud-2/>
- Microsoft Security. Shai-Hulud 2.0 Guidance. <https://www.microsoft.com/en-us/security/blog/2025/12/09/shai-hulud-2-0-guidance/>
- GitHub Blog. Strengthening Supply Chain Security. <https://github.blog/security/supply-chain-security/>

Comparative and Historical Context

- US GAO. SolarWinds Cyberattack Federal Response. <https://www.gao.gov/blog/solarwinds-cyberattack/>
- NCSC (UK). Log4j Vulnerability. <https://www.ncsc.gov.uk/information/log4j-vulnerability-what-everyone-needs-to-know>
- TeamPCP Interview (Erez Dasa, interviewer). ransomware-interviews.base44.app/interview/teampcp

PCPJack — Rival Worm

- SentinelLABS. PCPJack: Cloud Worm Evicts TeamPCP and Steals Credentials at Scale. <https://www.sentinelone.com/labs/cloud-worm-evicts-teampcp-and-steals-credentials-at-scale/>
- SecurityWeek. PCPJack Worm Removes TeamPCP Infections, Steals Credentials. <https://www.securityweek.com/pcpjack-worm-removes-teampcp-infections-steals-credentials/>
- BleepingComputer. New PCPJack Worm Steals Credentials, Cleans TeamPCP Infections. <https://www.bleepingcomputer.com/news/security/new-pcpjack-worm-steals-credentials-cleans-teampcp-infections/>
- GBHackers. PCPJack Worm Targets Docker, Kubernetes, Redis, and MongoDB Credentials. <https://gbhackers.com/pcpjack-worm-framework/>
- Infosecurity Magazine. PCPJack Campaign Boots TeamPCP Off Compromised Machines. <https://www.infosecurity-magazine.com/news/pcpjack-campaign-boots-teampcp-off/>

BreachForums Supply Chain Contest

- SOCRadar. BreachForums and TeamPCP Promote Supply Chain Competition. <https://socradar.io/blog/breachforums-teampcp-supply-chain-competition/>
- GBHackers. TeamPCP, BreachForums Launch \$1K Supply-Chain Attack Contest. <https://gbhackers.com/1k-supply-chain-attack-contest/>
- RedSide Security. TeamPCP and BreachForums Launch \$1,000 Contest to Drive Open-Source Supply Chain Attacks. <https://blog.redsidesecurity.com/post/teampcp-and-breachforums-launch-1000-contest-to-drive-open-source-supply-chain-attacks>

LAPSUS\$ Joint GitHub Data Sale

- The Hacker News. GitHub Breached — Employee Device Hack Led to Exfiltration of 3,800+ Internal Repos. <https://thehackernews.com/2026/05/github-investigating-teampcp-claimed.html>
- The Record (Recorded Future). GitHub Confirms Being Hacked by TeamPCP. <https://therecord.media/github-confirms-teampcp-hack-customers-unaffected>
- Help Net Security. TeamPCP Breached GitHub's Internal Codebase via Poisoned VS Code Extension. <https://www.helpnetsecurity.com/2026/05/20/github-breached-teampcp/>
- Infosecurity Magazine. GitHub Confirms Breach of Internal Repositories Via Malicious VS Code Extension. <https://www.infosecurity-magazine.com/news/github-confirms-breach-vs-code/>

Checkmarx April 2026 Additional Incidents

- Trend Micro. Analyzing TeamPCP's Supply Chain Attacks: Checkmarx KICS and elementary-data in CI/CD Credential Theft.
https://www.trendmicro.com/en_us/research/26/e/analyzing-teampcp-supply-chain-attacks.html
- SANS Internet Storm Center. TeamPCP Supply Chain Campaign: Activity Through 2026-05-17. <https://isc.sans.edu/diary/32994>

Additional Campaign Coverage

- Google Cloud Blog. Adversaries Leverage AI for Vulnerability Exploitation, Augmented Operations, and Initial Access.
<https://cloud.google.com/blog/topics/threat-intelligence/ai-vulnerability-exploitation-initial-access>
 - Okta Threat Intelligence. Defending Against TeamPCP Software Supply Chain Attacks.
https://www.okta.com/en-gb/blog/threat-intelligence/defending_against_team_pcp_software_supply_chain_attacks/
 - Security Boulevard. Mini Shai-Hulud: Frequently Asked Questions.
<https://securityboulevard.com/2026/05/mini-shai-hulud-frequently-asked-questions-about-the-teampcp-npm-and-pypi-supply-chain-campaign/>
-